

MI SYS API

1 Overview

1.1. Module Description

MI_SYS is the basic module of the whole MI system, which provides the basis for the operation of other MI modules.

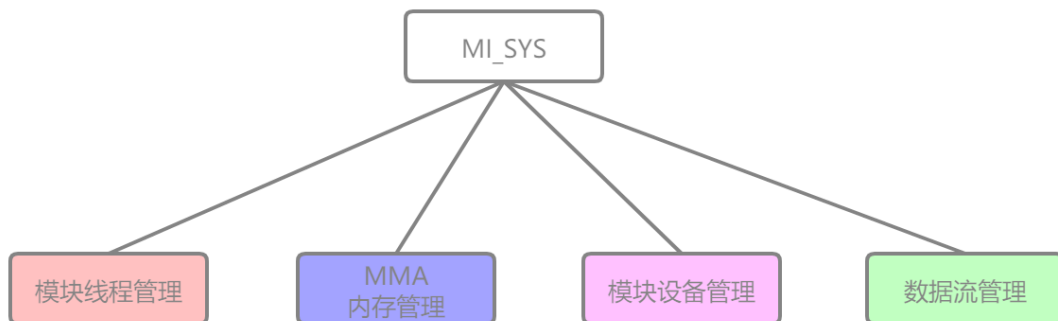


Figure 1-1 MI_SYS system frame diagram

As shown in Figure 1-1, the main functions of MI_SYS are summarized as follows:

- Realize MI system initialization, MMA memory buffer pool management.
- Provides a common interface for each module to register device nodes and establish a proc system.

- Provides an interface for each module to establish a binding relationship, and manages data flow between modules.
- Provide an interface for each module to apply for MMA continuous physical memory, manage memory allocation, map virtual addresses, and recycle memory.
- Provides an interface for each module to establish a worker thread, and manages the creation, operation, and destruction of each module thread.

The ko file corresponding to the MI_SYS module is mi_sys.ko, the library files are libmi_sys.so, libmi_sys.a, and the header files are mi_sys.h, mi_sys_datatype.h. For specific functions, please refer to the API interface description.

1.2. Document format constraints

Normal font: used for writing the body content of the document, in which the writing of the code segment needs to use a monospaced font.

Regular + Bold: used for writing important content in the body of the document.

Italics: Used for writing in the Tips section of the document.

Italic + bold: used for writing important content in the Tips section of the document.

1.3. Keyword Description

ID: Abbreviation for Identity document, which means unique code.

MI: Abbreviation of SStar SDK Middle Interface. In this article, if a structure similar to "MI_SYS" appears, it means the SYS module of MI. If it is simply "MI", it refers to the entire SDK.

Hex: Hexadecimal.

Kernl Mode: refers to the code working in the Kernel environment, and the code has the control authority to directly operate the hardware, such as functions and

threads in ko.

User Mode: refers to working in the User environment, such as customer applications, system calls, etc.

APP: Abbreviation for Application, here mainly refers to the application that calls the MI API.

API: Application Programming Interface, application programming interface.

NVR: The full name is Network Video Recorder, that is, network video recorder.

IPC: 全称IP Camera，即网络摄像机。

HW: 全称hardware，即硬件。

Dev: 全称Device，本文中表示MI模块设备，1.4.1有详细解释。

Chn: 全称Channel，本文中表示MI模块设备的某个通道，1.4.1有详细解释。

Port: 本文中表示MI模块设备通道中的某个端口，1.4.1有详细解释。

补充说明:

- 如无单独说明，本文中MI_SYS 和SYS，MI_DISP和DISP是相同意思，其余模块名称类似。
- 本文中出现的所有模块名称可以在[MI_ModuleId_e](#) [#321-mi_moduleid_e]中查阅其功能描述。

1.4. 流程框图

1.4.1. Dev/Chn/Port的关系

一个典型的MI 模块都会有Dev/chn/Port三级结构如图1-2。

Dev

一个MI 模块会有一个或多个Dev，一般来说不同的Dev表示该模块设备需要调用不同的HW资源，或者工作在不同的工作模式。比如说，VENC 在编码H264/H265时和编码Jpeg时，需要调用不同的HW资源，这个时候Dev就要分开了。

Chn

一个Dev 会有一个或多个Chn，Chn是通道的意思，一般来说不同的Chn表示该通道虽然和同Dev下的其他Chn共享HW资源，但是数据来源或者工作模式是不一样的。比如码流来源不同，Chn一般也不一样。

Port

一个Chn 会有一个或多个Port，Port是端口的意思，由InputPort和OutputPort组成。一般来说不同的Port表示该通道虽然和同Dev同Chn下的其他Port共享HW资源和数据来源，但是需要设置的参数是不一样的，比如分辨率不同，Port一般也不一样。

一般来说，Port才是客户操作MI模块的最小独立单位，因为它明确了所有的信息：**HW资源、数据来源、参数属性**。

每个Port都由InputPort和OutputPort组成，InputPort是数据流入的端口，OutputPort是数据流出的端口。但需要注意的是，一个Port并不是总是必须有InputPort和OutputPort的，这样取决于该模块的行为。比如Disp这样的模块就只需要InputPort，无需OutputPort，它直接把结果显示到了Panel上。Vdec这样的模块，数据来源可以是用户调用Vdec接口直接送入，不经过InputPort，这样他就没有OutputPort了。

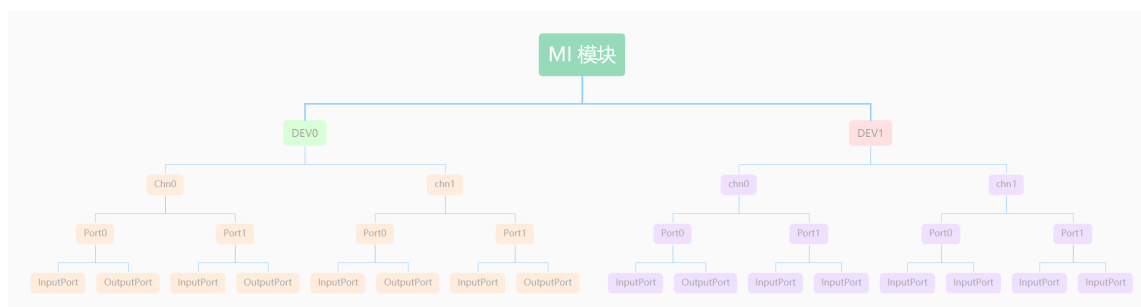


图 1-2 MI 模块的三级结构

Tips:

1. Chn和Port的界限并不是总是能区分清楚，如果某模块API文档的解释和上文的解释有差异，调用该模块请以它的API文档为准。
2. 一个Port的InputPort/OutputPort通常只有一个或零个，但某些特殊的模块例外。比如Vpe模块，它有一个InputPort和多个OutputPort。这表示Vpe共享同一个数据源，但是又必须要有不同规格的输出格式。

1.4.2. 一个典型的NVR数据流

图1-3是一个典型的NVR数据流模型。流动过程如下：

1. 建立Vdec->Divp->Disp的绑定关系；
2. 用户写入一段码流到Vdec的es buf；
3. Vdec解码，写入解码后的数据到Vdec OutputPort申请的内存，送入下一级；
4. Divp 接收数据，送到Divp HW进行处理，写入Divp OutputPort，送入下一级；
5. Disp 将接收到的数据显示出来。

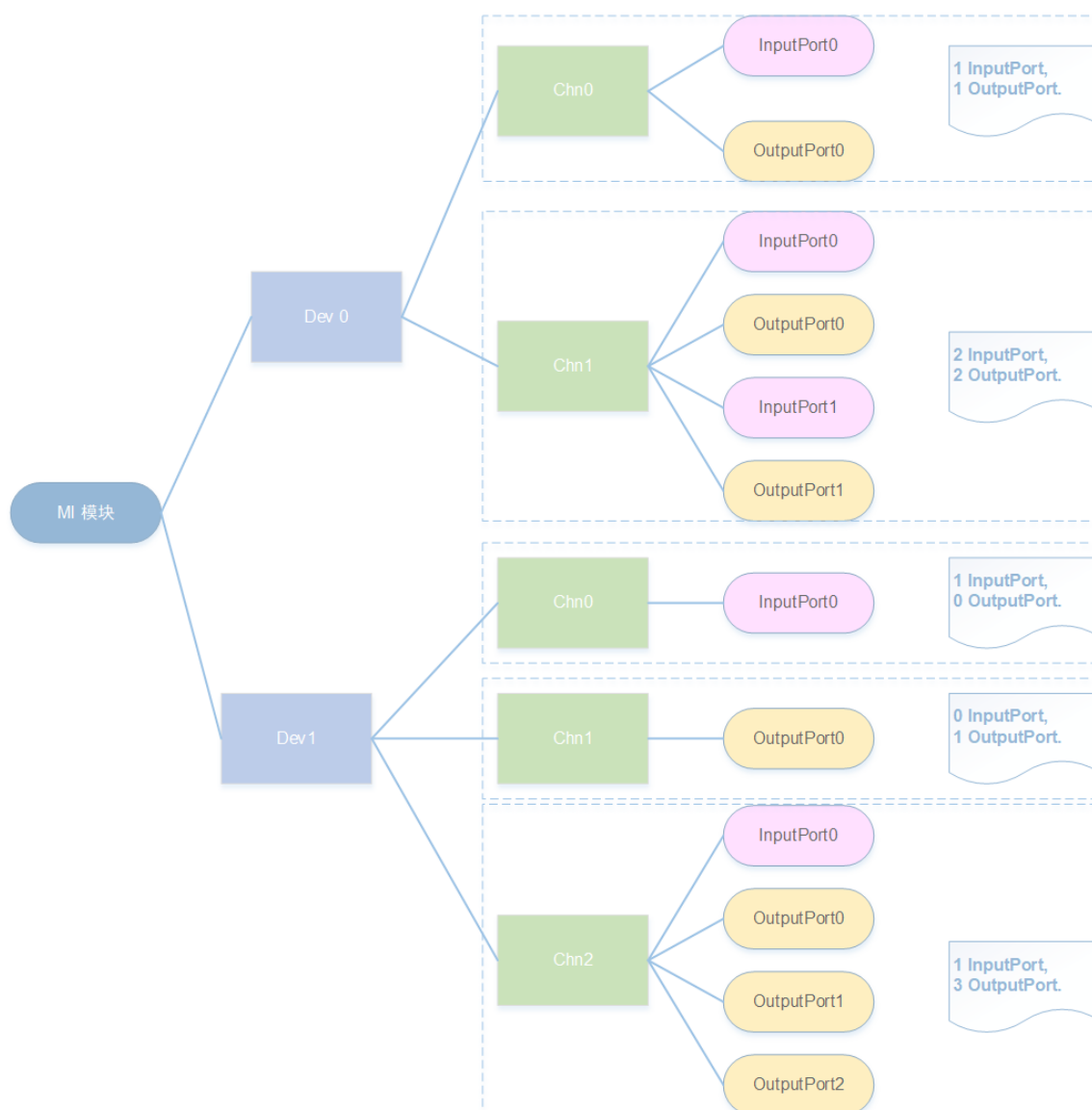


图 1-3 一个典型的NVR数据流模型

1.4.3. 一个典型的IPC数据流

图1-4是一个典型的IPC数据流模型，流动过程如下：

- 1. 建立Vif->Vpe->Venc的绑定关系；
- 2. Sensor 将数据送入vif处理；
- 3. Vif 将处理后的数据写入Output Port申请的内存，送入下一级；
- 4. Vpe接收数据，分别送入Pass0(ISP/SCL0)、Pass1(LDC)、Pass2(SCL1)进行处理，将处理的数据写入Output Port申请的内存，送入下一级；
- 5. Venc接收数据，送入编码器进行编码处理，将编码后的数据写入RingPool内存区；
- 6. 用户调用Venc的接口取流，送入用户业务层App。

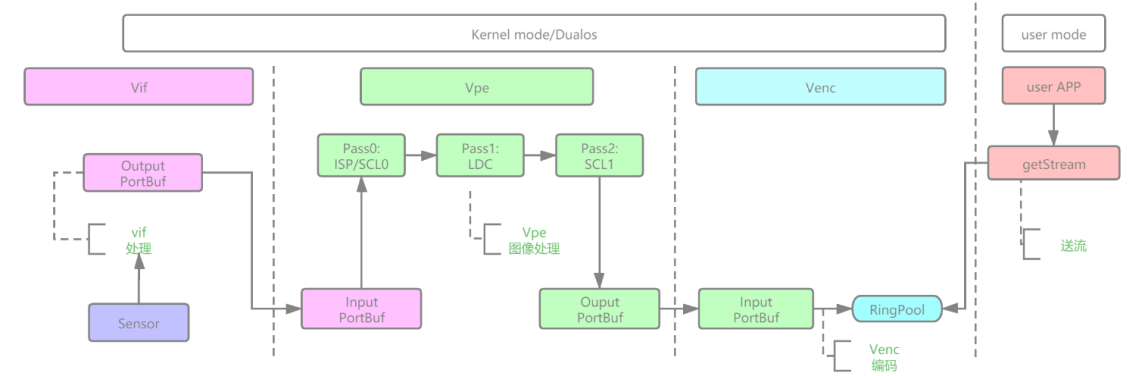


图 1-4 一个典型的IPC数据流模型

Tips:

Vpe内部的3个Pass之间的数据流动实际比图中要复杂一些。但这都是MI内部的处理逻辑，使用API时无需过度关注。

1.5. insmod参数说明

使用insmod mi_sys.ko 后面可以带的参数使用情况如下表所示。

表 1 1：insmod mi_sys.ko可加参数列表

参数	含义	取值
bEnableMmuPool	是否使能MMU	1:enable mmu 0:disable mmu Ex: bEnableMmuPool=1，表示使能mmu bEnableMmuPool=0，表示禁止mmu
logBufSize	缓存log的内存长度	默认为4Kbyte，单位：Byte Ex: logBufSize=4096，表示设置4096Byte，即4Kbyte
cmdQBufSize	cmdq 内存大小	由产品规格决定，单位：4Kbyte Ex: cmdQBufSize=256，表示设置256Kbyte

.....

2. API 参考

.....

2.1. API描述格式说明

本手册使用 9个参考域描述 API 的相关信息，它们作用如 表所表示。

标签	功能
功能	简要描述API的主要功能。
语法	列出调用API应包括的头文件以及API的原型声明。
参数	列出API的参数、参数说明及参数属性。
返回值	列出API所有可能的返回值及其含义。
依赖	列出API包含的头文件和API编译时要链接的库文件。
注意	列出使用API时应注意的事项。
举例	列出使用API的实例。
相关主题	调用上下文中有关联的接口。

2.2. 功能模块API列表

如前文所述，我们可以粗略的把MI_SYS的API分为三大类：系统功能类、数据流类、内存管理类。

API名	功能
系统功能类	
MI_SYS_Init [#231-mi_sys_init]	初始化MI_SYS系统
MI_SYS_Exit [#232-mi_sys_exit]	析构MI_SYS系统
MI_SYS_GetVersion [#233-mi_sys_getversion]	获取MI 的系统版本号
MI_SYS_GetCurPts [#234-mi_sys_getcurpts]	获取MI系统当前时间戳
MI_SYS_InitPtsBase [#235-	初始化MI系统基准时间戳

mi_sys_initptsbase]	
MI_SYS_SyncPts [#236-mi_sys_syncpts]	同步MI系统时间戳
MI_SYS_SetReg [#237-mi_sys_setreg]	设置寄存器的值，调试用
MI_SYS_GetReg [#238-mi_sys_getreg]	获取寄存器的值，调试用
MI_SYS_ReadUuid [#239-mi_sys_readuuid]	获取Chip的Unique ID
MI_SYS_EnableChnOutputPortLowLatency [#2310-mi_sys_enablechnoutputportlowlatency]	打开或者关闭通道output端口的低延迟输出功能
MI_SYS_InitDev [#2311-mi_sys_initdev]	初始化sys设备
MI_SYS_DeInitDev [#2312-mi_sys_deinitdev]	反初始化sys设备
MI_SYS_SetGlobalFlag [#2313-mi_sys_setglobalflag]	设置全局标志位的值
数据流类	
MI_SYS_BindChnPort [#241-mi_sys_bindchnport]	数据源输出端口到接收者输入端口的绑定
MI_SYS_BindChnPort2 [##242-mi_sys_bindchnport2]	数据源输出端口到接收者输入端口的绑定，需要指定工作模式
MI_SYS_UnBind_ChnPort [#243-mi_sys_unbind_chnport]	数据源输出端口到接收者输入端口的解绑定
MI_SYS_GetBindbyDest [#244-mi_sys_getbindbydest]	查询数据接收者输入端口的对应的源输出端口
MI_SYS_ChnInputPortGetBuf [#245-mi_sys_chninputportgetbuf]	获取通道inputPort的buf
MI_SYS_ChnInputPortPutBuf [#246-mi_sys_chninputportputbuf]	将通道inputPort的buf加到待处理队列功能

MI_SYS_ChnOutputPortGetBuf [#247-mi_sys_chnoutputportgetbuf]	获取通道outputPort的buf
MI_SYS_ChnOutputPortPutBuf [#248-mi_sys_chnoutputportputbuf]	释放通道outputPort的buf
MI_SYS_ChnInputPortGetBufPa [#249-mi_sys_chninputportgetbufpa]	分配通道input端口对应的buf object，仅提供MIU物理地址
MI_SYS_ChnInputPortPutBufPa [#2410-mi_sys_chninputportputbufpa]	把通道input端口对应的buf object加到待处理队列，与 MI_SYS_ChnInputPortGetBufPa [#249-mi_sys_chninputportgetbufpa]成对使用
MI_SYS_ChnOutputPortGetBufPa [#2411-mi_sys_chnoutputportgetbufpa]	分配通道output端口对应的buf object，仅提供MIU物理地址
MI_SYS_ChnOutputPortPutBufPa [#2412-mi_sys_chnoutputportputbufpa]	释放通道output端口对应的buf object，与 MI_SYS_ChnOutputPortGetBufPa [#2411-mi_sys_chnoutputportgetbufpa]成对使用
MI_SYS_SetChnOutputPortDepth [#2413-mi_sys_setchnoutputportdepth]	设置通道OutputPort的深度
MI_SYS_ChnPortInjectBuf [#2414-mi_sys_chnportinjectbuf]	向模块通道inputPort注入outputPort Buf数据
MI_SYS_GetFd [#2415-mi_sys_getfd]	获取当前通道等待事件的文件描述符
MI_SYS_CloseFd [#2416-mi_sys_closefd]	关闭当前通道的文件描述符
MI_SYS_DupBuf [#2417-mi_sys_dupbuf]	复制一个buf object
内存管理类	
MI_SYS_SetChnMMAConf [#251-mi_sys_setchnmmaconf]	设置模块设备通道输出端口默认分配内存的MMA池名称
MI_SYS_GetChnMMAConf [#252-mi_sys_getchnmmaconf]	获取模块设备通道输出端口默认分配内存的MMA池名称
MI_SYS_ConfDevPubPools [#253-mi_sys_confdevpubpools]	配置并初始化模块公共缓冲池

MI_SYS_ReleaseDevPubPools [#254-mi_sys_releasedevpubpools]	释放模块公共缓冲池
MI_SYS_ConfGloPubPools [#255-mi_sys_confglopubpools]	配置并初始化MI全系统默认VB缓存池
MI_SYS_ReleaseGloPubPools [#256-mi_sys_releaseglopubpools]	释放MI全系统默认VB缓存池
MI_SYS_MMA_Alloc [#257-mi_sys_mma_alloc]	应用程序从MMA内存管理池申请物理连续内存
MI_SYS_MMA_Free [#258-mi_sys_mma_free]	用户态释放从MMA内存管理池中分配的内存
MI_SYS_Mmap [#259-mi_sys_mmap]	映射物理内存到CPU虚拟地址
MI_SYS_FlushInvCache [#2510-mi_sys_flushinvcache]	Flush cache CPU虚拟地址
MI_SYS_Munmap [#2511-mi_sys_munmap]	取消物理内存到虚拟地址的映射
MI_SYS_MemsetPa [#2512-mi_sys_memsetpa]	通过DMA硬件模块，填充整块物理内存
MI_SYS_MemcpyPa [#2513-mi_sys_memcpyPa]	通过DMA硬件模块，把源内存数据拷贝到目标内存上
MI_SYS_BufFillPa [#2514-mi_sys_buffillpa]	通过DMA硬件模块，填充部分物理内存
MI_SYS_BufBlitPa [#2515-mi_sys_bufblitpa]	通过DMA硬件模块，把源内存数据上的部分区域拷贝到目标内存上的部分区域
MI_SYS_ConfigPrivateMMAPool [#2516-mi_sys_configprivatemmapool]	为模块配置私有MMA Heap
MI_SYS_PrivateDevChnHeapAlloc [#2517-mi_sys_privatedevchnheapalloc]	从模块通道私有MMA Pool 申请内存
API名 MI_SYS_PrivateDevChnHeapFree [#2518-	功能 从模块通道私有MMA pool 释放内存

mi_sys_privatedevchnheapfree]	
MI_SYS_Va2Pa [#2519-mi_sys_va2pa]	将MI内存块的CPU虚拟地址转成内存物理地址
MI_SYS_GetMmaPoolInfo [#2520-mi_sys_getmmapoolinfo]	获取指定mma 池的总空闲大小和物理上连续的最大的内存长度

2.3. 系统功能类API

2.3.1. MI_SYS_Init

- 功能

MI_SYS初始化，MI_SYS 模块为系统内其它MI模组提供基础支援，需要早于系统内其它MI模组初始化，否则其它stream类型的模组初始化时会失败。

- 语法

```
MI_S32 MI_SYS_Init(void);
```

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- [MI_SYS_Init](#) [#231-mi_sys_init]需要早于其他MI模组的Init函数调用。
- 可以在同一进程或多进程中重复调用[MI_SYS_Init](#) [#231-mi_sys_init]，但必须和[MI_SYS_Exit](#) [#232-mi_sys_exit]成对使用，否则会报错。
- 系统在内核启动参数内，需要配置好MMA 内存堆的配置参数。

- 举例

系统Init调用Sample

```
MI_S32 ST_Sys_Init(void)
{
    MI_SYS_Version_t stVersion;
    MI_U64 u64Pts = 0;

    STCHECKRESULT(MI_SYS_Init());

    memset(&stVersion, 0x0, sizeof(MI_SYS_Version_t));
    STCHECKRESULT(MI_SYS_GetVersion(&stVersion));
    ST_INFO("u8Version:%s\n", stVersion.u8Version);

    STCHECKRESULT(MI_SYS_GetCurPts(&u64Pts));
    ST_INFO("u64Pts:0x%llx\n", u64Pts);

    u64Pts = 0xF1237890F1237890;
    STCHECKRESULT(MI_SYS_InitPtsBase(u64Pts));

    u64Pts = 0xE1237890E1237890;
    STCHECKRESULT(MI_SYS_SyncPts(u64Pts));

    return MI_SUCCESS;
}

MI_S32 ST_Sys_Exit(void)
{
    STCHECKRESULT(MI_SYS_Exit());

    return MI_SUCCESS;
}
```

Tips:

本示例适用于: [MI_SYS_Init](#) [#231-mi_sys_init] / [MI_SYS_Exit](#) [#232-mi_sys_exit] / [MI_SYS_GetVersion](#) [#233-mi_sys_getversion] / [MI_SYS_GetCurPts](#) [#234-mi_sys_getcurpts] / [MI_SYS_InitPtsBase](#) [#235-mi_sys_initptsbase] / [MI_SYS_SyncPts](#) [#236-mi_sys_syncpts] / [MI_SYS_ReadUuid](#) [#239-mi_sys_readuuid] .

- 相关主题

[MI_SYS_Exit](#) [#232-mi_sys_exit]

.....

2.3.2. MI_SYS_Exit

- 功能

MI_SYS初始化，调用[MI_SYS_Exit](#) [#232-mi_sys_exit]之前，需要确保系统内所有其他模组都已经完成去初始化，所有VBPOOL都已经销毁，否则[MI_SYS_Exit](#) [#232-mi_sys_exit]会返回失败。

- 语法

```
MI_S32 MI_SYS_Exit (void);
```

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- [MI_SYS_Exit](#) [#232-mi_sys_exit]调用前需要确保系统内所有其他模组都已经完成去初始化。
- [MI_SYS_Exit](#) [#232-mi_sys_exit]调用前需要确保系统内所有创建的VBPOOL已经成功销毁。

- 举例

参见 [系统Init调用Sample](#) [#jump4]举例。

- 相关主题

[MI_SYS_Init](#) [#231-mi_sys_init]

2.3.3. MI_SYS_GetVersion

- 功能

获取MI 的系统版本号。

- 语法

```
MI_S32 MI_SYS_GetVersion (MI_SYS_Version_t *pstVersion);
```

- 形参

参数名称	描述	输入/输出
pstVersion	系统版本号返回数据结构指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 举例

参见 [系统Init调用Sample](#) [jump4]举例。

2.3.4. MI_SYS_GetCurPts

- 功能

获取MI系统当前时间戳。

- 语法

```
MI_S32 MI_SYS_GetCurPts (MI_U64 *pu64Pts);
```

- 形参

参数名称	描述	输入/输出
pu64Pts	系统当前时间戳返回地址	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 举例

参见[系统Init调用Sample](#) [#jump4]举例。

2.3.5. MI_SYS_InitPtsBase

- 功能
初始化MI系统时间戳基准。
- 语法

```
MI_S32 MI_SYS_InitPtsBase (MI_U64 u64PtsBase);
```

- 形参

参数名称	描述	输入/输出
u64PtsBase	设置的系统时间戳基准	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 举例

参见[系统Init调用Sample](#) [#jump4]举例。

2.3.6. MI_SYS_SyncPts

- 功能
微调同步MI系统时间戳。
- 语法

```
MI_S32 MI_SYS_SyncPts (MI_U64 u64Pts);
```

- 形参

参数名称	描述	输入/输出
u64Pts	微调后的系统时间戳基准	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 举例

参见[系统Init调用Sample](#) [#jump4]举例。

2.3.7. MI_SYS_SetReg

- 功能
设置寄存器的值，调试用。
- 语法

```
MI_S32 MI_SYS_SetReg (MI_U32 u32RegAddr, MI_U16 u16Value,  
MI_U16 u16Mask);
```

• 形参

参数名称	描述	输入/输出
u32RegAddr	Register 总线之地址	输入
u16Value	待写入之16bit寄存器值	输入
u16Mask	本次写入寄存器值之Mask遮挡栏位	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so

2.3.8. MI_SYS_GetReg

- 功能

读取寄存器的值，调试用。
- 语法

```
MI_S32 MI_SYS_GetReg (MI_U32 u32RegAddr, MI_U16 *pu16Value);
```

• 形参

参数名称	描述	输入/输出
u32RegAddr	Register 总线之地址	输入
pu16Value	待读回16bit寄存器值返回地址	输出

- 返回值

- 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
 - 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
-

2.3.9. MI_SYS_ReadUuid

- 功能
获取Chip的Unique ID
- 语法

```
MI_S32 MI_SYS_ReadUuid (MI_U64 *u64Uuid);
```

- 形参

参数名称	描述	输入/输出
u64Uuid	获取chip unique ID值的指针	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
 - 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
 - 举例
参见[系统Init调用Sample](#) [#jump4]举例。
-

2.3.10. MI_SYS_EnableChnOutputPortLowLatency

- 功能

打开或者关闭通道output端口的低延迟输出功能

- 语法

```
MI_S32 MI_SYS_EnableChnOutputPortLowLatency(MI_SYS_ChnPort_t
*pstChnPort,MI_BOOL bEnable , MI_U32 u32Param);
```

- 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之output端口的指针	输入
bEnable	TRUE：打开；FALSE：关闭。默认FALSE	输入
u32Param	Low Latency参数，其含义由具体模块决定 VPE用于设置Line Count，表示写完多少条Line后就把Output port buf给用户或后级	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

当bEnable 为true时，u32Param必须大于0，否则设置无效

- 举例

```
MI_SYS_ChnPort_t stChnPort;

stChnPort.eModId = E_MI_MODULE_ID_VPE;

stChnPort.u32DevId = 0;

stChnPort.u32ChnId = 0;
```



```

stChnPort.u32PortId = 0;

//打开VPE 通道1 output端口0 的低延迟输出功能,设定完成100行即可输出
MI_SYS_EnableChnOutputPortLowLatency(&stChnPort,TRUE , 100);

//关闭VPE 通道1 output端口0 的低延迟输出功能,
MI_SYS_EnableChnOutputPortLowLatency(&stChnPort,FALSE , 0);

```

2.3.11. MI_SYS_InitDev

- 功能

MI_SYS初始化，MI_SYS 模块为系统内其它MI模组提供基础支援，需要早于系统内其它MI 模组初始化，否则其它stream类型的模组初始化时会失败。

- 语法

```
MI_S32 MI_SYS_InitDev(MI_SYS_InitParam_t *pstInitParam);
```

- 形参

参数名称	描述	输入/输出
pstInitParam	设备初始化参数	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- [MI_SYS_InitDev](#) [#2311-mi_sys_initdev]需要早于其他MI模组的Init函数调用。
- 系统在内核启动参数内，需要配置好MMA 内存堆的配置参数。

- pstInitParam暂未使用，空值传入即可。
- 此接口在Version 2.17以上版本推荐使用，用于替换原有MI_SYS_Init [#231-mi_sys_init]接口。

- 相关主题

[MI_SYS_DeInitDev](#) [#2312-mi_sys_deinitdev]

2.3.12. MI_SYS_DeInitDev

- 功能

MI_SYS初始化，调用[MI_SYS_DeInitDev](#) [#2312-mi_sys_deinitdev]之前，需要确保系统内所有其他模组都已经完成去初始化，所有VBPOOL都已经销毁，否则[MI_SYS_DeInitDev](#) [#2312-mi_sys_deinitdev] 会返回失败。

- 语法

```
MI_S32 MI_SYS_DeInitDev(void);
```

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 此函数必须在初始化设备后调用，否则返回失败。
- 此接口同个进程下不支持多次调用，否则会返回失败。
- [MI_SYS_DeInitDev](#) [#2312-mi_sys_deinitdev]调用前需要确保系统内所有其他模组都已经完成去初始化。
- 此接口在Version 2.17以上版本推荐使用，用于替换原有[MI_SYS_Exit](#) [#232-mi_sys_exit]接口。

- 相关主题

2.3.13. MI_SYS_SetGlobalFlag

- 功能

设置全局标志位的值。通过MI_SYS_SetGlobalFlag设置全局标志位的值来告诉MI_SYS IR灯已打开，目前只支持设置IR标志位的值。

- 语法

```
MI_S32 MI_SYS_SetGlobalFlag(MI_SYS_GlobalFlagParam_t
*pstGlobalFlagParam);
```

- 形参

参数名称	描述	输入/输出
pstGlobalFlagParam	设置全局Flag参数	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

当IR灯打开时，需要通过MI_SYS_SetGlobalFlag接口将MI_SYS_GlobalFlagParam_t中的value成员设置为1，这样在后面通过MI_SYS_ChnOutputPortGetBuf 获取到的stBufInfo里面的u32IrFlag成员同样会记录为1。

若MI_SYS_SetGlobalFlag接口在中断中被调用，则需要将MI_SYS_GlobalFlagParam_t参数中的blsr参数设置为1。

2.4. 数据流类 API

2.4.1. MI_SYS_BindChnPort

- 功能

数据源输出端口到数据接收者输入端口的绑定。

- 语法

```
MI_S32 MI_SYS_BindChnPort( *pstSrcChnPort, *pstDstChnPort,
,MI_U32 u32SrcFrmrate, MI_U32 u32DstFrmrate);
```

- 形参

参数名称	参数含义	输入/输出
pstSrcChnPort	源端口配置信息数据结构指针	输入
pstDstChnPort	目标端口配置信息数据结构指针	输入
u32SrcFrmrate	源端口配置的帧率	输入
u32DstFrmrate	目标端口配置的帧率	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 源端口必须是通道输出端口。
- 目标端口必须是通道输入端口。

- 源和目标端口必须之前没有被绑定过。
- 本接口只支持按照E_MI_SYS_BIND_TYPE_FRAME_BASE方式绑定模块，在Version 2.0以上版本不推荐使用，请使用[MI_SYS_BindChnPort2](#) [##242-mi_sys_bindchnport2]替代。
- 举例

```
MI_SYS_ChnPort_t stSrcChnPort;

MI_SYS_ChnPort_t stDstChnPort;

MI_U32 u32SrcFrmrate;

MI_U32 u32DstFrmrate;

stSrcChnPort.eModId = E_MI_MODULE_ID_VPE;

stSrcChnPort.u32DevId = 0;

stSrcChnPort.u32ChnId = 0;

stSrcChnPort.u32PortId = 0;

stDstChnPort.eModId = E_MI_MODULE_ID_VENC;

stDstChnPort.u32DevId = 0;

stDstChnPort.u32ChnId = 0;

stDstChnPort.u32PortId = 0;

u32SrcFrmrate = 30;

u32DstFrmrate = 30;

MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, u32SrcFrmrate, u
```

- 相关主题

[MI_SYS_BindChnPort2](#) [##242-mi_sys_bindchnport2]

[MI_SYS_UnBind_ChnPort](#) [#243-mi_sys_unbind_chnport]

.....

2.4.2. MI_SYS_BindChnPort2

- 功能

数据源输出端口到数据接收者输入端口的绑定，需要额外指定工作模式。

- 语法

```
MI_S32 MI_SYS_BindChnPort2(MI_SYS_ChnPort_t *pstSrcChnPort,
MI_SYS_ChnPort_t *pstDstChnPort,MI_U32 u32SrcFrmrate, MI_U32
u32DstFrmrate, MI_SYS_BindType_e eBindType, MI_U32
u32BindParam);
```

- 形参

参数名称	参数含义	输入/输出
pstSrcChnPort	源端口配置信息数据结构指针。	输入
pstDstChnPort	目标端口配置信息数据结构指针。	输入
u32SrcFrmrate	源端口配置的帧率	输入
u32DstFrmrate	目标端口配置的帧率	输入
eBindType	源端口与目标端口连接的工作模式，参考 MI_SYS_BindType_e [#MI_SYS_BindType_e]	输入
u32BindParam	不同工作模式需带入的额外参数。	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 源端口必须是通道输出端口。

- 目标端口必须是通道输入端口。
- 同一个源端口可以绑定多个目标端口。
- 同一个目标端口只能绑定一个源端口，使用之前必须没有被绑定过。
- MI_SYS_BindChnPort2在pstSrcChnPort、pstDstChnPort和eBindType相同的情况下可以被多次调用，来重新设置需要修改的u32SrcFrmrate和u32DstFrmrate。
- 旧版本MI SYS不提供此接口，如没有找到，不用设置。
- 各种eBindType使用场景如下：

eBindType	适用场景
E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY	u32BindParam表示低时延值，单位ms
E_MI_SYS_BIND_TYPE_HW_RING	u32BindParam表示ring buffer depth，目前是只有vpe和venc (h264/h265) 支持这种模式，只支持一路; vif->isp，支持一路；isp->scl，支持多路
E_MI_SYS_BIND_TYPE_REALTIME	u32BindParam未使用，jpe imi会走这种模式，只支持一路
E_MI_SYS_BIND_TYPE_FRAME_BASE	u32BindParam未使用，默认是走这种frame mode

- 举例

```

MI_SYS_ChnPort_t stSrcChnPort;
MI_SYS_ChnPort_t stDstChnPort;
MI_U32 u32SrcFrmrate;
MI_U32 u32DstFrmrate;
MI_SYS_BindType_e eBindType;
MI_U32 u32BindParam;

// a.    vpe与venc连接方式为E_MI_SYS_BIND_TYPE_FRAME_BASE，代码
如下：
stSrcChnPort.eModId = E_MI_MODULE_ID_VPE;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;

```

```

    stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32PortId = 0;
    u32SrcFrmrate = 30;
    u32DstFrmrate = 30;
    eBindType = E_MI_SYS_BIND_TYPE_FRAME_BASE;
    u32BindParam = 0;
    STCHECKRESULT(MI_SYS_BindChnPort2(&stSrcChnPort, &stDstChnPort,
    u32SrcFrmrate, u32DstFrmrate, eBindType, u32BindParam));

// b.   vpe与jpe连接方式为E_MI_SYS_BIND_TYPE_REALTIME · 代码如下：
    stSrcChnPort.eModId = E_MI_MODULE_ID_VPE;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32PortId = 0;
    stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
    stDstChnPort.u32DevId = 1;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32PortId = 0;
    u32SrcFrmrate = 30;
    u32DstFrmrate = 30;
    eBindType = E_MI_SYS_BIND_TYPE_REALTIME;
    u32BindParam = 0;
    STCHECKRESULT(MI_SYS_BindChnPort2(&stSrcChnPort, &stDstChnPort,
    u32SrcFrmrate, u32DstFrmrate, eBindType, u32BindParam));

// c.   vpe与venc连接方式为E_MI_SYS_BIND_TYPE_HW_RING · 代码如下：
    stSrcChnPort.eModId = E_MI_MODULE_ID_VPE;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32PortId = 0;
    stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32PortId = 0;
    u32SrcFrmrate = 30;
    u32DstFrmrate = 30;
    eBindType = E_MI_SYS_BIND_TYPE_HW_RING;
    u32BindParam = 1080;//假设vpe output resolution为1920*1080 ·
    设置ring buffer depth为1080
    STCHECKRESULT(MI_SYS_BindChnPort2(&stSrcChnPort,
    &stDstChnPort, u32SrcFrmrate, u32DstFrmrate, eBindType,
    u32BindParam));

```

- 相关主题

[MI_SYS_BindChnPort2](#) [##242-mi_sys_bindchnport2]

2.4.3. MI_SYS_UnBind_ChnPort

- 功能

数据源输出端口到数据接收者输入端口之间的解绑定。

- 语法

```
MI_S32 MI_SYS_UnBindChnPort(*pstSrcChnPort, *pstDstChnPort);
```

- 形参

参数名称	描述	输入/输出
pstSrcChnPort	源端口配置信息数据结构指针	输入
pstDstChnPort	目标端口配置信息数据结构指针	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 源端口必须是通道输出端口。
- 目标端口必须是通道输入端口。
- 源和目标端口之间之前必须已经被绑定过

- 相关主题

[MI_SYS_BindChnPort](#) [#241-mi_sys_bindchnport]

[MI_SYS_BindChnPort2](#) [##242-mi_sys_bindchnport2]

2.4.4. MI_SYS_GetBindbyDest

- 功能

查询数据接收者输入端口的对应的源输出端口。

- 语法

```
MI_S32 MI_SYS_GetBindbyDest (MI_SYS_ChnPort_t *pstDstChnPort,
MI_SYS_ChnPort_t *pstSrcChnPort);
```

- 形参

参数名称	描述	输入/输出
pstDstChnPort	目标端口配置信息数据结构指针	输入
pstSrcChnPort	源端口配置信息数据结构指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 目标端口必须是通道输入端口。
- 目标端口之前必须已经被绑定过

2.4.5. MI_SYS_ChnInputPortGetBuf

- 功能

分配通道input端口对应的buf object。

• 语法

```
MI_S32 MI_SYS_ChnInputPortGetBuf (MI_SYS_ChnPort_t *pstChnPort,
MI_SYS_BufConf_t *pstBufConf, MI_SYS_BufInfo_t *pstBufInfo,
MI_SYS_BUF_HANDLE *phHandle, MI_S32 s32TimeOutMs);
```

• 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之input端口的指针	输入
pstBufConf	待分配内存配置信息	输入
pstPortBuf	返回之buf 指针	输出
phHandle	获取的intputPort Buf的handle	输出
s32TimeOutMs	等待超时的毫秒数，>=0时为具体时间，<0时会采用默认值20ms	输入

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

• 举例

MI_SYS_ChnInputPortGetBuf调用Sample

```
MI_SYS_ChnPort_t stVpeChnInput;

MI_SYS_BUF_HANDLE hHandle = 0;

MI_SYS_BufConf_t stBufConf;

MI_SYS_BufInfo_t stBufInfo;
```

```

**struct** timeval stTv;

MI_U16 u16Width = 1920, u16Height = 1080;

FILE *fp = NULL;

memset(&stVpeChnInput, 0x0, sizeof(MI_SYS_ChnPort_t));

memset(&stBufConf, 0x0, sizeof(MI_SYS_BufConf_t));

memset(&stBufInfo, 0x0, sizeof(MI_SYS_BufInfo_t));

stVpeChnInput.eModId = E_MI_MODULE_ID_VPE;

stVpeChnInput.u32DevId = 0;

stVpeChnInput.u32ChnId = 0;

stVpeChnInput.u32PortId = 0;

fp = fopen("/mnt/vpeport0_1920x1080_pixel0_737.raw", "rb");

if(fp == NULL)
{
    printf("file %s open fail\\n", "/mnt/vpeport0_1920x1080_p

    return 0;
}

while(1)
{
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;

    gettimeofday(&stTv, NULL);

    stBufConf.u64TargetPts = stTv.tv_sec*1000000 + stTv.tv_us

    stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV42

    stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN

```

```

        stBufConf.stFrameCfg.u16Width = u16Width;

        stBufConf.stFrameCfg.u16Height = u16Height;

        if(MI_SUCCESS == MI_SYS_ChnInputPortGetBuf(&stVpeChnInpu

    {

        if(fread(stBufInfo.stFrameData.pVirAddr[0], u16Width*
<= 0)

        {

            fseek(fp, 0, SEEK_SET);

        }

        MI_SYS_ChnInputPortPutBuf(hHandle, &stBufInfo, FALSE);

    }

}

```

- 相关主题

[MI_SYS_ChnInputPortPutBuf](#) [#246-mi_sys_chninputportputbuf]

[MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

2.4.6. MI_SYS_ChnInputPortPutBuf

- 功能

把通道input端口对应的buf object加到待处理队列。

- 语法

```

MI_S32 MI_SYS_ChnInputPortPutBuf (MI_SYS_BUF_HANDLE hHandle ,
    *pstPortBuf , MI_BOOL bDropBuf);

```

- 形参

参数名称	描述	输入/输出
hHandle	当前buf的Handle	输入
pstPortBuf	待提交的buf 指针	输入
bDropBuf	是否将buf直接丢弃而不用提交到待处理队列上	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so

- 举例

参见[MI_SYS_ChnInputPortGetBuf调用Sample](#) [#jump1]举例。

- 相关主题

[MI_SYS_ChnInputPortGetBuf](#) [#245-mi_sys_chninputportgetbuf]

[MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

.....

2.4.7. MI_SYS_ChnOutputPortGetBuf

- 功能

获取通道output端口输出buf对应buf object。
- 语法

```
MI_S32 MI_SYS_ChnOutputPortGetBuf (*pstChnPort, *pstBufInfo ,
MI_SYS_BUF_HANDLE *phHandle);
```

- 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之output端口的指针	输入
pstBufInfo	返回之buf 指针	输出
phHandle	获取的outputPort Buf的handle	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 举例

MI_SYS_ChnOutputPortGetBuf调用Sample

```

MI_SYS_ChnPort_t stChnPort;
MI_SYS_BufInfo_t stBufInfo;
MI_SYS_BUF_HANDLE stBufHandle;
MI_S32 s32Ret = MI_SUCCESS;
MI_S32 s32Fd = 0;
fd_set read_fds;
struct timeval TimeoutVal;
char szFileName[128];
int fd = 0;
MI_U32 u32GetFramesCount = 0;
MI_BOOL _bWriteFile = TRUE;

stChnPort.eModId = E_MI_MODULE_ID_DIVP;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = DIVP_CHN_FOR_VDF;
stChnPort.u32PortId = 0;

s32Ret = MI_SYS_GetFd(&stChnPort, &s32Fd);
if(MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_GetFd 0, error, %X\n", s32Ret);
    return NULL;
}

```

```

}
s32Ret = MI_SYS_SetChnOutputPortDepth(&stChnPort, 2, 3);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortDepth err:%x,
chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}

sprintf(szFileName, "divp%d.es", stChnPort.u32ChnId);
printf("start to record %s\n", szFileName);
fd = open(szFileName, O_RDWR | O_CREAT | O_TRUNC, S_IRUSR |
S_IWUSR | S_IRGRP | S_IROTH);
if (fd < 0)
{
    ST_ERR("create %s fail\n", szFileName);
}

while (1)
{
    FD_ZERO(&read_fds);
    FD_SET(s32Fd, &read_fds);

    TimeoutVal.tv_sec = 1;
    TimeoutVal.tv_usec = 0;

    s32Ret = select(s32Fd + 1, &read_fds, NULL, NULL,
&TimeoutVal);

    if(s32Ret < 0)
    {
        ST_ERR("select failed!\n");
        // usleep(10 * 1000);
        continue;
    }
    else if(s32Ret == 0)
    {
        ST_ERR("get divp frame time out\n");
        //usleep(10 * 1000);
        continue;
    }
    else
    {
        if(FD_ISSET(s32Fd, &read_fds))
        {
            s32Ret = MI_SYS_ChnOutputPortGetBuf(&stChnPort,
&stBufInfo, &stBufHandle);

```



```

        if(MI_SUCCESS != s32Ret)
        {
            //ST_ERR("MI_SYS_ChnOutputPortGetBuf err,
%x\n", s32Ret);
            continue;
        }

        // save one Frame YUV data
        if (fd > 0)
        {
            if(_bWriteFile)
            {
                write(fd,
stBufInfo.stFrameData.pVirAddr[0],
stBufInfo.stFrameData.u16Height *
stBufInfo.stFrameData.u32Stride[0] +
                stBufInfo.stFrameData.u16Height *
stBufInfo.stFrameData.u32Stride[1] /2);
            }

        }

        ++u32GetFramesCount;
        printf("channelId[%u] u32GetFramesCount[%u]\n",
stChnPort.u32ChnId, u32GetFramesCount);

        MI_SYS_ChnOutputPortPutBuf(stBufHandle);
    }
}

if (fd > 0)
{
    close(fd);
    fd = -1;
}
}

```

```

MI_SYS_SetChnOutputPortDepth(&stChnPort, 0, 3);
printf("exit record\n");
return NULL;

```

- 相关主题

[MI_SYS_ChnOutputPortPutBuf](#) [#248-mi_sys_chnoutputportputbuf]

[MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

2.4.8. MI_SYS_ChnOutputPortPutBuf

- 功能

释放通道output端口输出buf对应的buf object。

- 语法

```
MI_S32 MI_SYS_ChnOutputPortPutBuf (MI_SYS_BUF_HANDLE
hBufHandle);
```

- 形参

参数名称	描述	输入/输出
hBufHandle	待提交的buf 的handle	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 举例

参见[MI_SYS_ChnOutputPortGetBuf](#)调用[Sample](#) [#jump2] 举例。

- 相关主题

- [MI_SYS_ChnOutputPortGetBuf](#) [#247-mi_sys_chnoutputportgetbuf]
- [MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

2.4.9. MI_SYS_ChnInputPortGetBufPa

- 功能

分配通道input端口对应的buf object，仅提供MIU物理地址。

- 语法

```
MI_S32 MI_SYS_ChnInputPortGetBufPa (MI_SYS_ChnPort_t
*pstChnPort, MI_SYS_BufConf_t *pstBufConf, MI_SYS_BufInfo_t
*pstBufInfo, MI_SYS_BUF_HANDLE *phHandle , MI_S32
s32TimeOutMs);
```

- 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之input端口的指针	输入
pstBufConf	待分配内存配置信息	输入
pstPortBuf	返回之buf 指针	输出
phHandle	获取的inputPort Buf的 handle	输出
s32TimeOutMs	等待超时的毫秒数，>=0时为具体时间；<0时会采用默认值20ms	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

与MI_SYS_ChnInputPortGetBuf区别：MI_SYS_ChnInputPortGetBufPa获取的pstBufInfo中只能使用物理地址phyAddr，而MI_SYS_ChnInputPortGetBuf获取的pstBufInfo中物理地址phyAddr和虚拟地址pVirAddr都可以使用。

- 举例

MI_SYS_ChnInputPortGetBufPa调用Sample

```

MI_SYS_ChnnPort_t stVpeChnInput;
MI_SYS_BUF_HANDLE hHandle = 0;
MI_SYS_BufConf_t stBufConf;
MI_SYS_BufInfo_t stBufInfo;
struct timeval stTv;
MI_U16 u16Width = 1920, u16Height = 1080;
FILE *fp = NULL;
void *pVirAddr = NULL;

memset(&stVpeChnInput, 0x0, sizeof(MI_SYS_ChnnPort_t));
memset(&stBufConf, 0x0, sizeof(MI_SYS_BufConf_t));
memset(&stBufInfo, 0x0, sizeof(MI_SYS_BufInfo_t));

stVpeChnInput.eModId = E_MI_MODULE_ID_VPE;
stVpeChnInput.u32DevId = 0;
stVpeChnInput.u32ChnId = 0;
stVpeChnInput.u32PortId = 0;

fp = fopen("/mnt/vpeport0_1920x1080_pixel0_737.raw", "rb");
if (fp == NULL)
{
    printf("file %s open fail\n",
"/mnt/vpeport0_1920x1080_pixel0_737.raw");
    return 0;
}

while (1)
{
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
    gettimeofday(&stTv, NULL);
    stBufConf.u64TargetPts = stTv.tv_sec * 1000000 +
stTv.tv_usec;
    stBufConf.stFrameCfg.eFormat =
E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
    stBufConf.stFrameCfg.eFrameScanMode =
E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
    stBufConf.stFrameCfg.u16Width = u16Width;
    stBufConf.stFrameCfg.u16Height = u16Height;

    if (MI_SUCCESS ==
MI_SYS_ChnnInputPortGetBufPa(&stVpeChnInput, &stBufConf,
&stBufInfo, &hHandle, 0))
    {
        pVirAddr =
MI_SYS_Mmap(stBufInfo.stFrameData.phyAddr[0],
stBufInfo.stFrameData.u32BufSize, &pVirAddr, TRUE);
        assert(pVirAddr);
        if (fread(pVirAddr, u16Width * u16Height * 2, 1, fp) <=

```

```

0)
    {
        fseek(fp, 0, SEEK_SET);
    }
    MI_SYS_Munmap(pVirAddr,
stBufInfo.stFrameData.u32BufSize);
    MI_SYS_ChnInputPortPutBufPa(hHandle, &stBufInfo,
FALSE);
    }
}

```

- 相关主题

[MI_SYS_ChnInputPortPutBufPa](#) [#2410-mi_sys_chninputportputbufpa]

[MI_SYS_ChnInputPortPutBuf](#) [#246-mi_sys_chninputportputbuf]

[MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

2.4.10. MI_SYS_ChnInputPortPutBufPa

- 功能

把通道input端口对应的buf object加到待处理队列，与

[MI_SYS_ChnInputPortGetBufPa](#) [#249-mi_sys_chninputportgetbufpa]成对使用。

- 语法

```

MI_S32 MI_SYS_ChnInputPortPutBufPa (MI_SYS_BUF_HANDLE hHandle
· MI_SYS_BufInfo_t *pstPortBuf · MI_BOOL bDropBuf);

```

- 形参

参数名称	描述	输入/输出
hHandle	当前buf的 Handle	输入
pstPortBuf	待提交的buf 指针	输入
bDropBuf	直接放弃对buf的修改不提交	输入

- 返回值

- 0 成功。
- 非0 失败，参照错误码。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

与MI_SYS_ChnInputPortPutBuf区别：MI_SYS_ChnInputPortPutBufPa提交的pstBufInfo中只能使用物理地址phyAddr，而MI_SYS_ChnInputPortPutBuf提交的pstBufInfo中物理地址phyAddr和虚拟地址pVirAddr都可以使用

- 举例

参见 [MI_SYS_ChnInputPortGetBufPa](#) [#249-mi_sys_chninputportgetbufpa]调用Sample 举例。

- 相关主题

[MI_SYS_ChnInputPortGetBufPa](#) [#249-mi_sys_chninputportgetbufpa]

[MI_SYS_ChnInputPortGetBuf](#) [#245-mi_sys_chninputportgetbuf]

[MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

2.4.11. MI_SYS_ChnOutputPortGetBufPa

- 功能

获取通道output端口输出buf对应的buf object，仅提供MIU物理地址。

- 语法

```
MI_S32 MI_SYS_ChnOutputPortGetBufPa (MI_SYS_ChnPort_t
    *pstChnPort, MI_SYS_BufInfo_t *pstBufInfo, MI_SYS_BUF_HANDLE
    *phHandle);
```

- 形参

|参数名称|描述|输入/输出| |pstChnPort|指向模块通道之output端口的指针|
输入| |pstBufInfo|返回之buf 指针|输出| |phHandle|获取的outputPort Buf的
handle|输出|

- 返回值

- 0 成功。

- 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so

- 注意

与MI_SYS_ChnOutputPortGetBuf区别：MI_SYS_ChnOutputPortGetBufPa获取的pstBufInfo中只能使用物理地址phyAddr，而MI_SYS_ChnOutputPortGetBuf获取的pstBufInfo中物理地址phyAddr和虚拟地址pVirAddr都可以使用。

- 举例

```
MI_SYS_ChnPort_t stChnPort;
MI_SYS_BufInfo_t stBufInfo;
MI_SYS_BUF_HANDLE stBufHandle;
MI_S32 s32Ret = MI_SUCCESS;
MI_S32 s32Fd = 0;
fd_set read_fds;
struct timeval TimeoutVal;
char szFileName[128];
int fd = 0;
MI_U32 u32GetFramesCount = 0;
MI_BOOL _bWriteFile = TRUE;
void *pVirAddr = NULL;

stChnPort.eModId = E_MI_MODULE_ID_DIVP;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = DIVP_CHN_FOR_VDF;
stChnPort.u32PortId = 0;

s32Ret = MI_SYS_GetFd(&stChnPort, &s32Fd);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_GetFd 0, error, %X\n", s32Ret);
    return NULL;
}
s32Ret = MI_SYS_SetChnOutputPortDepth(&stChnPort, 2, 3);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortDepth err:%x,
chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}
```

```

}

sprintf(szFileName, "divp%d.es", stChnPort.u32ChnId);
printf("start to record %s\n", szFileName);
fd = open(szFileName, O_RDWR | O_CREAT | O_TRUNC, S_IRUSR |
S_IWUSR | S_IRGRP | S_IROTH);
if (fd < 0)
{
    ST_ERR("create %s fail\n", szFileName);
}

while (1)
{
    FD_ZERO(&read_fds);
    FD_SET(s32Fd, &read_fds);

    TimeoutVal.tv_sec = 1;
    TimeoutVal.tv_usec = 0;

    s32Ret = select(s32Fd + 1, &read_fds, NULL, NULL,
&TimeoutVal);

    if (s32Ret < 0)
    {
        ST_ERR("select failed!\n");
        // usleep(10 * 1000);
        continue;
    }
    else if (s32Ret == 0)
    {
        ST_ERR("get divp frame time out\n");
        //usleep(10 * 1000);
        continue;
    }
    else
    {
        if (FD_ISSET(s32Fd, &read_fds))
        {
            s32Ret = MI_SYS_ChnOutputPortGetBufPa(&stChnPort,
&stBufInfo, &stBufHandle);

            if (MI_SUCCESS != s32Ret)
            {
                //ST_ERR("MI_SYS_ChnOutputPortGetBufPa err,
%x\n", s32Ret);
                continue;
            }

            s32Ret =

```



```

MI_SYS_Mmap(stBufInfo.stFrameData.phyAddr[0],
stBufInfo.stFrameData.u32BufSize, &pVirAddr, TRUE);
    assert(pVirAddr);
    // save one Frame YUV data
    if (fd > 0)
    {
        if (_bWriteFile)
        {
            write(fd, pVirAddr,
stBufInfo.stFrameData.u16Height *
stBufInfo.stFrameData.u32Stride[0] +
stBufInfo.stFrameData.u16Height *
stBufInfo.stFrameData.u32Stride[1] / 2);
        }

        ++u32GetFramesCount;
        printf("channelId[%u] u32GetFramesCount[%u]\n",
stChnPort.u32ChnId, u32GetFramesCount);

        MI_SYS_Munmap(pVirAddr,
stBufInfo.stFrameData.u32BufSize);
        MI_SYS_ChnOutputPortPutBufPa(stBufHandle);
    }
}

if (fd > 0)
{
    close(fd);
    fd = -1;
}

MI_SYS_SetChnOutputPortDepth(&stChnPort, 0, 3);
printf("exit record\n");
return NULL;
MI_SYS_ChnOutputPortGetBufPa调用Sample

```

- 相关主题

[MI_SYS_ChnOutputPortPutBufPa](#) [#2412-mi_sys_chnoutputportputbufpa]

[MI_SYS_ChnOutputPortPutBuf](#) [#248-mi_sys_chnoutputportputbuf]

[MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

2.4.12. MI_SYS_ChnOutputPortPutBufPa

- 功能

释放通道output端口输出buf对应的buf object，与[MI_SYS_ChnOutputPortGetBufPa](#) [#2411-mi_sys_chnoutputportgetbufpa]成对使用。

- 语法

```
MI_S32 MI_SYS_ChnOutputPortPutBufPa (MI_SYS_BUF_HANDLE hBufHandle);
```

- 形参

参数名称	描述	输入/输出
hBufHandle	待提交的buf的handle	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

与MI_SYS_ChnOutputPortPutBuf区别：MI_SYS_ChnOutputPortPutBufPa提交的handle对应的pstBufInfo中只能使用物理地址phyAddr，而MI_SYS_ChnOutputPortPutBuf提交的handle对应的pstBufInfo中物理地址phyAddr和虚拟地址pVirAddr都可以使用。

- 举例

参见 [MI_SYS_ChnOutputPortGetBufPa](#) [#2411-mi_sys_chnoutputportgetbufpa]调用Sample 举例。

- 相关主题

[MI_SYS_ChnOutputPortGetBufPa](#) [#2411-mi_sys_chnoutputportgetbufpa]
[MI_SYS_ChnOutputPortGetBuf](#) [#247-mi_sys_chnoutputportgetbuf]

2.4.13. MI_SYS_SetChnOutputPortDepth

• 功能

设置通道output端口对应的系统buf 数量和用户可以拿到的buf数量。

• 语法

```
MI_S32 MI_SYS_SetChnOutputPortDepth(MI_SYS_ChnPort_t
    *pstChnPort, MI_U32 u32UserFrameDepth, MI_U32
    u32BufQueueDepth);
```

• 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之output端口的指针	输入
u32UserFrameDepth	设置该output用户可以拿到的buf最大数量	输入
u32BufQueueDepth	设置该output系统buf最大数量	输入

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

• 注意

- 在使用完OutputPortBuf (MI_SYS_ChnOutputPortGetBuf / MI_SYS_ChnOutputPortPutBuf 不再需要被调用) 之后，建议把 u32UserFrameDepth设置为0，可以减少output buf的缓存个数，节省内存。

- u32UserFrameDepth 必须小于或者等于u32BufQueueDepth。
- 当u32UserFrameDepth等于u32BufQueueDepth，并且user fifo queue缓存的个数等于u32BufQueueDepth时，当前output port将会停止工作。
- 如果output port 没有绑定后级，想让当前output port工作，必须设置u32UserFrameDepth 大于0。
- 如果output port 通过realtime mode或者ring mode方式绑定，当前output port的u32UserFrameDepth必须设置为0。
- 其他使用限制可参考vif/isp/scl/venc/jpd API介绍。
- 举例

参见[MI_SYS_ChnOutputPortGetBuf调用Sample](#) [#jump2] 举例。

2.4.14. MI_SYS_ChnPortInjectBuf

- 功能
把获取的outputPort buf插到指定的inputPort Buf Queue中
- 语法

```
MI_S32 MI_SYS_ChnPortInjectBuf(MI_SYS_BUF_HANDLE hHandle
,MI_SYS_ChnPort_t *pstChnInputPort);
```

- 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之input端口的指针	输入
hHandle	获取的outputPort Buf的handle	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so
- 注意
调用MI_SYS_ChnPortInjectBuf 后，user不能再显示release handle。
- 举例
参见 MI_SYS_ChnOutputPortGetBuf调用Sample [#jump2] 举例。
- 相关主题
[MI_SYS_ChnInputPortGetBuf](#) [#245-mi_sys_chninputportgetbuf]
[MI_SYS_ChnInputPortPutBuf](#) [#246-mi_sys_chninputportputbuf]
[MI_SYS_ChnOutputPortGetBuf](#) [#247-mi_sys_chnoutputportgetbuf]
[MI_SYS_ChnOutputPortPutBuf](#) [#248-mi_sys_chnoutputportputbuf]

2.4.15. MI_SYS_GetFd

- 功能
获取当前output Port等待事件的文件描述符
- 语法

```
MI_S32 MI_SYS_GetFd(MI_SYS_ChnPort_t *pstChnPort , MI_S32 *ps32Fd);
```

- 形参

参数名称	描述	输入/输出
pstChnPort	端口信息结构体指针	输入
ps32Fd	等待事件的文件描述符	输出

- 返回值
 - 0 成功。

- 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 注意
 - 需要与[MI_SYS_CloseFd](#) [#2416-mi_sys_closefd] 成对使用。
 - 推荐使用使用fd + select 的方式取数据，这样MI_SYS只有在fd对应的port 口有数据的时候才会唤醒线程，效率比采用while+sleep循环取数据的效率要高。
- 举例

参见[MI_SYS_ChnOutputPortGetBuf调用Sample](#) [#jump2] 举例。
- 相关主题

[MI_SYS_CloseFd](#) [#2416-mi_sys_closefd]

2.4.16. MI_SYS_CloseFd

- 功能

关闭当前通道的文件描述符
- 语法

```
MI_S32 MI_SYS_CloseFd(MI_S32 s32ChnPortFd);
```

- 形参

参数名称	描述	输入/输出
s32ChnPortFd	等待事件的文件描述符	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。

- 依赖
 - 头文件：mi_sys_datatype.h 、 mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 注意

需要与[MI_SYS_GetFd](#) [#2415-mi_sys_getfd] 成对使用。
- 举例

参见[MI_SYS_ChnOutputPortGetBuf调用Sample](#) [#jump2] 举例。
- 相关主题

[MI_SYS_GetFd](#) [#2415-mi_sys_getfd]

2.4.17. MI_SYS_DupBuf

- 功能

复制一个buf object。
- 语法

```
MI_S32 MI_SYS_DupBuf(MI_SYS_BUF_HANDLE srcBufHandle ,
MI_SYS_BUF_HANDLE *pDupTargetBufHandle);
```

- 形参

参数名称	描述	输入/输出
srcBufHandle	源buf object	输入
pDupTargetBufHandle	返回复制的buf object 指针	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so
- 举例

MI_SYS_DupBuf调用Sample

```
int test0()
{
    MI_SYS_ChnPort_t stChnPort;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BUF_HANDLE bufHandle ,DupTargetBufHandle;
    MI_S32 s32Ret;
    stChnPort.eModId = E_MI_MODULE_ID_VPE;
    stChnPort.u32DevId = 0;
    stChnPort.u32ChnId = 0;
    stChnPort.u32PortId = 0;
    s32Ret = MI_SYS_ChnOutputPortGetBuf
(&stChnPort,&stBufInfo,&bufHandle);
    if(s32Ret != MI_SUCCESS)
        return 0;
    s32Ret = MI_SYS_DupBuf(bufHandle, &DupTargetBufHandle);
    if(s32Ret != MI_SUCCESS)
    {
        MI_SYS_ChnOutputPortPutBuf(bufHandle);
        return 0;
    }
    stChnPort.eModId = E_MI_MODULE_ID_DIVP;
    stChnPort.u32DevId = 0;
    stChnPort.u32ChnId = 0;
    stChnPort.u32PortId = 0;
    MI_SYS_ChnPortInjectBuf(DupTargetBufHandle , &stChnPort);

    *****
    可以继续操作stBufInfo
    *****

    MI_SYS_ChnOutputPortPutBuf(bufHandle);
    return 1;
}

int test1()
{
    MI_SYS_ChnPort_t stChnPort;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BufConf_t stBufConf;
    MI_SYS_BUF_HANDLE bufHandle ,DupTargetBufHandle;
    MI_S32 s32Ret;
```



```

    stChnPort.eModId = E_MI_MODULE_ID_DIVP;
    stChnPort.u32DevId = 0;
    stChnPort.u32ChnId = 0;
    stChnPort.u32PortId = 0;
    memset(&stBufConf, 0, sizeof(stBufConf));
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
    stBufConf.stFrameCfg.eFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422;
    stBufConf.stFrameCfg.u16Height = 1080;
    stBufConf.stFrameCfg.u16Width = 1920;
    stBufConf.stFrameCfg.eFrameScanMode =
E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
    s32Ret = MI_SYS_ChnInputPortGetBuf
(&stChnPort, &stBufConf, &stBufInfo, &bufHandle, 0);
    if(s32Ret != MI_SUCCESS)
        return 0;
    *****
    填充stBufInfo
    *****
    s32Ret = MI_SYS_DupBuf(bufHandle, &DupTargetBufHandle);
    if(s32Ret != MI_SUCCESS)
    {
        MI_SYS_ChnInputPortPutBuf(bufHandle);
        return 0;
    }
    MI_SYS_ChnInputPortPutBuf(DupTargetBufHandle, &stChnPort,
FALSE);
    *****
    可以继续操作stBufInfo
    *****

    MI_SYS_ChnInputPortPutBuf(bufHandle, &stChnPort, FALSE);
    return 1;
}

```

- 相关主题

[MI_SYS_ChnOutputPortGetBuf](#) [#247-mi_sys_chnoutputportgetbuf]

[MI_SYS_ChnOutputPortPutBuf](#) [#248-mi_sys_chnoutputportputbuf]

[MI_SYS_ChnInputPortGetBuf](#) [#245-mi_sys_chninputportgetbuf]

[MI_SYS_ChnInputPortPutBuf](#) [#246-mi_sys_chninputportputbuf]

[MI_SYS_ChnPortInjectBuf](#) [#2414-mi_sys_chnportinjectbuf]

2.5. 内存管理类 API

2.5.1. MI_SYS_SetChnMMAConf

- 功能
设置模块设备通道输出默认分配内存的MMA池名称。
- 语法

```
MI_S32 MI_SYS_SetChnMMAConf (MI_ModuleId_e eModId, MI_U32 u32DevId, MI_U32 u32ChnId, MI_U8 *pu8MMAHeapName);
```

- 形参

参数名称	描述	输入/输出
eModId	待配置的模块ID	输入
u32DevId	待配置的设备ID	输入
u32ChnId	待配置的通道号	输入
pu8MMAHeapName	设置MMA heap name	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 注意
若不调用MI_SYS_SetChnMMAConf接口设置或者设置pu8MMAHeapName值为NULL时，则使用默认的MMA池名称mma_heap_name0。
- 举例

```
MI_ModuleId_e eVifModeId = E_MI_MODULE_ID_VIF;
MI_VIF_DEV vifDev = 0;
MI_VIF_CHN vifChn = 0;

MI_SYS_SetChnMMAConf(eVifModeId, vifDev, vifChn,
"mma_heap_name0");
```

2.5.2. MI_SYS_GetChnMMAConf

- 功能

获取模块设备通道输出端口默认分配内存的MMA池名称。

- 语法

```
MI_S32 MI_SYS_GetChnMMAConf (MI_ModuleId_e eModId, MI_U32
u32DevId, MI_U32 u32ChnId, void * pu8MMAHeapName, MI_U32
u32Length);
```

- 形参

参数名称	描述	输入/输出
eModId	待配置的模块ID	输入
u32DevId	待配置的设备ID	输入
u32ChnId	待配置的通道号	输入
pu8MMAHeapName	要获取的MMA heap name	输入
u32Length	pu8MMAHeapName 指向内存的长度，最大为64Byte	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so
- 相关主题

[MI_SYS_SetChnMMAConf](#) [#251-mi_sys_setchnmmaconf]

2.5.3. MI_SYS_ConfDevPubPools

- 功能
配置并初始化模块公共缓冲池。
- 语法

```
MI_S32 MI_SYS_ConfDevPubPools(MI_ModuleId_e eModule, MI_U32 u32DevId, MI_VB_PoolListConf_t stPoolListConf);
```

- 形参

参数名称	描述	输入/输出
eModule	目标模块ID	输入
u32DevId	Dev ID	输入
stPoolListConf	模块公共缓冲池队列配置	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 注意

Version 2.0及以上版本默认不开启，推荐使用
[MI_SYS_ConfigPrivateMMAPool](#) [#mi_sys_configprivatemmapool]。

2.5.4. MI_SYS_ReleaseDevPubPools

- 功能
释放全局的公共缓冲池。
- 语法

```
MI_S32 MI_SYS_ReleaseDevPubPools(MI_ModuleId_e eModule, MI_U32 u32DevId);
```

- 形参

参数名称	描述	输入/输出
eModule	目标模块ID	输入
u32DevId	Dev ID	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 举例

Version 2.0及以上版本默认不开启，推荐使用
[MI_SYS_ConfigPrivateMMAPool](#) [#2516-mi_sys_configprivatemmapool]。

2.5.5. MI_SYS_ConfGloPubPools

- 功能

配置并初始化系统全局公共缓冲池。

- 语法

```
MI_S32 MI_SYS_ConfGloPubPools(MI_VB_PoolListConf_t
stPoolListConf);
```

- 形参

参数名称	描述	输入/输出
stPoolListConf	模块公共缓冲池队列配置	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

Version2.0及以上版本默认不开启，推荐使用[MI_SYS_ConfigPrivateMMAPool](#) [#2516-mi_sys_configprivatemmapool]。

2.5.6. MI_SYS_ReleaseGloPubPools

- 功能

释放全局的公共缓冲池。

- 语法

```
MI_S32 MI_VB_ ReleaseGloPubPools (void);
```

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h 、 mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 举例

Version 2.0及以上版本默认不开启，推荐使用
[MI_SYS_ConfigPrivateMMAPool](#) [#2516-mi_sys_configprivatemmapool]。

2.5.7. MI_SYS_MMA_Alloc

- 功能
直接向MMA内存管理器申请分配内存。
- 语法

```
MI_S32 MI_SYS_MMA_Alloc(MI_U8 *pstMMAHeapName, MI_U32 u32BlkSize ,MI_PHY *phyAddr);
```

- 形参

参数名称	描述	输入/输出
pstMMAHeapName	目标MMA heapname	输入
u32BlkSize	待分配的块字节大小	输入
phyAddr	返回的内存块物理地址	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so
- 注意

返回的内存块物理地址与实际分配的内存长度都是4K对齐。

- 举例

MI_SYS_MMA_Alloc调用Sample

```
MI_PHY phySrcBufAddr = 0;
void *pVirSrcBufAddr = NULL;
MI_U32 srcBuffSize = 1920 * 1980 * 3 / 2;
srcBuffSize = ALIGN_UP(srcBuffSize, 4096);

ret = MI_SYS_MMA_Alloc(NULL, srcBuffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
    return -1;
}

ret = MI_SYS_Mmap(phySrcBufAddr, srcBuffSize, &pVirSrcBufAddr,
TRUE);
if(ret != MI_SUCCESS)
{
    MI_SYS_MMA_Free(phySrcBufAddr);
    printf("mmap src buff failed\n");
    return -1;
}

memset(pVirSrcBufAddr, 0, srcBuffSize);
MI_SYS_FlushInvCache(pVirSrcBufAddr, srcBuffSize);
MI_SYS_Munmap(pVirSrcBufAddr, srcBuffSize);
MI_SYS_MMA_Free(phySrcBufAddr);
```

- 相关主题

[MI_SYS_MMA_Free](#) [#258-mi_sys_mma_free]

[MI_SYS_Mmap](#) [#259-mi_sys_mmap]

[MI_SYS_FlushInvCache](#) [#2510-mi_sys_flushinvcache]

[MI_SYS_Munmap](#) [#2511-mi_sys_munmap]

2.5.8. MI_SYS_MMA_Free

- 功能

直接向MMA内存管理器释放之前分配的内存。

- 语法

```
MI_S32 MI_SYS_MMA_Free(MI_U64 phyAddr);
```

- 形参

参数名称	描述	输入/输出
phyAddr	待释放之内存的物理地址	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 举例

参见[MI_SYS_MMA_Alloc调用Sample](#) [jump3]举例。

- 相关主题

[MI_SYS_MMA_Alloc](#) [#257-mi_sys_mma_alloc]

[MI_SYS_Mmap](#) [#259-mi_sys_mmap]

[MI_SYS_FlushInvCache](#) [#2510-mi_sys_flushinvcache]

[MI_SYS_Munmap](#) [#2511-mi_sys_munmap]

2.5.9. MI_SYS_Mmap

- 功能

映射任意物理内存到当前用户态进程的 CPU 虚拟地址空间。

- 语法

```
MI_S32 MI_SYS_Mmap(MI_U64 u64PhyAddr, MI_U32 u32Size , void  
**ppVirtualAddress , MI_BOOL bCache);
```

- 形参

参数名称	描述	输入/输出
u64PhyAddr	待映射的物理地址	输入
u32Size	待映射的物理地址长度	输入
ppVirtualAddress	存储CPU虚拟地址指针的指针	输出
bCache	是否map成cache还是un-cache	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件：mi_sys_datatype.h 、 mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 物理地址是SStar内存控制器地址。
- 物理地址必须4Kbyte对齐。
- 物理地址长度必须4Kbyte对齐。
- 物理内存必须完整的落在MMA管理的内存范围内或者linux kenrel管理的内存之外。

- 举例

参见 [MI_SYS_MMA_Alloc调用Sample](#) [jump3] 举例。

- 相关主题

[MI_SYS_MMA_Alloc](#) [#257-mi_sys_mma_alloc]

[MI_SYS_MMA_Free](#) [#258-mi_sys_mma_free]

[MI_SYS_FlushCache](#) [#mi_sys_flushinvcache]

[MI_SYS_Munmap](#) [#2511-mi_sys_munmap]

2.5.10. MI_SYS_FlushInvCache

- 功能

Flush cache 。

- 语法

```
MI_S32 MI_SYS_FlushCache(MI_VOID *pVirtualAddress, MI_U32 u32Size);
```

- 形参

参数名称	描述	输入/输出
pVirtualAddress	之前 MI_SYS_Mmap [#259-mi_sys_mmap] 返回的 CPU 虚拟地址	输入
u32Size	待flush的cache长度	输入

- 返回值

- 0 成功 。
- 非0 失败，参照[错误码](#) [#jump] 。

- 依赖

- 头文件：mi_sys_datatype.h 、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 待flush cache的物理地址必须4Kbyte对齐 。

- 待flush cache的映射长度必须4Kbyte对齐。
- 待flush cache的映射内存范围必须是之前通过MI_SYS_Mmap [#259-mi_sys_mmap] API获取到的。
- 待flush cache的映射内存应该是MI_SYS_Mmap [#259-mi_sys_mmap]时采用cache方式进行的，nocache的方式无需flush cache。
- 举例

参见 MI_SYS_MMA_Alloc调用Sample [#jump3]举例。

- 相关主题

MI_SYS_MMA_Alloc [#257-mi_sys_mma_alloc]

MI_SYS_MMA_Free [#258-mi_sys_mma_free]

MI_SYS_Mmap [#259-mi_sys_mmap]

MI_SYS_Munmap [#2511-mi_sys_munmap]

2.5.11. MI_SYS_Munmap

- 功能
取消物理内存到虚拟地址的映射。
- 语法

```
MI_S32 MI_SYS_UnMmap(MI_VOID *pVirtualAddress, MI_U32 u32Size);
```

- 形参

参数名称	描述	输入/输出
pVirtualAddress	之前MI_SYS_Mmap [#259-mi_sys_mmap] 返回的CPU 虚拟地址	输入
u32Size	待取消映射的映射长度	输入

- 返回值
 - 0 成功。

- 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 注意
 - 待取消映射的虚拟地址必须4Kbyte对齐。
 - 待取消映射的映射长度必须4Kbyte对齐
 - 待取消的映射内存范围必须是之前通过[MI_SYS_Mmap](#) [#259-mi_sys_mmap] API获取到的。
- 举例

参见[MI_SYS_MMA_Alloc](#)调用[Sample](#) [#jump3] 举例。
- 相关主题

[MI_SYS_MMA_Alloc](#) [#257-mi_sys_mma_alloc]

[MI_SYS_MMA_Free](#) [#258-mi_sys_mma_free]

[MI_SYS_Mmap](#) [#259-mi_sys_mmap]

[MI_SYS_FlushInvCache](#) [#2510-mi_sys_flushinvcache]

2.5.12. MI_SYS_MemsetPa

- 功能

通过DMA硬件模块，填充整块物理内存.
- 语法

```
MI_S32 MI_SYS_MemsetPa(MI_PHY phyPa, MI_U32 u32Val, MI_U32
u32Lenth);
```

- 形参

参数名称	参数含义	输入/输出
phyPa	填充的物理地址	输入
u32Val	填充值	输入
u32Lenth	填充大小，单位为byte	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 举例

MI_SYS_MemsetPa调用Sample

```

MI_PHY phySrcBufAddr = 0;
MI_PHY phyDstBufAddr = 0;
void *pVirSrcBufAddr = NULL;
void *pVirDstBufAddr = NULL;
MI_U32 buffSize = 1920 * 1980 * 3 / 2;
buffSize = ALIGN_UP(buffSize, 4096);

ret = MI_SYS_MMA_Alloc(NULL, buffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
    return -1;
}

ret = MI_SYS_MMA_Alloc(NULL, buffSize, &phyDstBufAddr);
if(ret != MI_SUCCESS)
{
    MI_SYS_MMA_Free(phySrcBufAddr);
    printf("alloc dts buff failed\n");
    return -1;
}

```

```
MI_SYS_MemsetPa(phySrcBufAddr, 0xff, buffSize);
MI_SYS_MemsetPa(phyDstBufAddr, 0x00, buffSize);
MI_SYS_MemcpyPa(phyDstBufAddr, phySrcBufAddr, buffSize);
MI_SYS_MMA_Free(phySrcBufAddr);
MI_SYS_MMA_Free(phyDstBufAddr);
```

- 相关主题

[MI_SYS_MemcpyPa](#) [#2513-mi_sys_memcpyPa]

2.5.13. MI_SYS_MemcpyPa

- 功能

通过DMA硬件模块，把源内存数据拷贝到目标内存上.

- 语法

```
MI_S32 MI_SYS_MemcpyPa(MI_PHY phyDst, MI_PHY phySrc, MI_U32
u32Lenth);
```

- 形参

参数名称	参数含义	输入/输出
phyDst	目的物理地址	输入
phySrc	源物理地址	输入
u32Lenth	拷贝大小，单位为byte	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 举例

参见[MI_SYS_MemsetPa](#) [#2512-mi_sys_memsetpa] 举例。

- 相关主题

[MI_SYS_MemsetPa](#) [#2512-mi_sys_memsetpa]

2.5.14. MI_SYS_BufFillPa

- 功能

通过DMA硬件模块，填充部分物理内存。

- 语法

```
MI_S32 MI_SYS_BufFillPa(MI_SYS_FrameData_t *pstBuf, MI_U32 u32Val, MI_SYS_WindowRect_t *pstRect);
```

- 形参

参数名称	参数含义	输入/输出
pstBuf	填充的帧数据描述之结构体	输入
u32Val	填充值	输入
pstRect	填充的数据范围	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- `pstRect`数据范围是以`pstBuf`所描述的首地址为（0，0）点开始，宽高都是以`pstBuf`中的`ePixelFormat`来计算每一个pixel所移动的内存数据大小来进行部分填充。
- `pstBuf`中`u16Width`、`u16Height`、`phyAddr`、`u32Stride`、`ePixelFormat`为必填，其余数值无意义。
- 举例

```
MI_S32 ret = 0;
MI_SYS_WindowRect_t rect;
MI_SYS_FrameData_t stSysFrame;
memcpy(&stSysFrame, &buf->stFrameData,
sizeof(MI_SYS_FrameData_t));

if(pRect) {
    rect.u16X = pRect->left;
    rect.u16Y = pRect->top;
    rect.u16Height = pRect->bottom-pRect->top;
    rect.u16Width = pRect->right-pRect->left;
} else {
    rect.u16X = 0;
    rect.u16Y = 0;
    rect.u16Height = stSysFrame.u16Height;
    rect.u16Width = stSysFrame.u16Width;
}

DBG_INFO("rect %d %d %d %d \n", rect.u16X, rect.u16Y
        , rect.u16Width, rect.u16Height);
ret = MI_SYS_BufBlitPa(&stSysFrame, u32ColorVal, &rect);

return ret;
```

- 相关主题

[MI_SYS_BufBlitPa](#) [#2515-mi_sys_bufblitpa]

2.5.15. MI_SYS_BufBlitPa

- 功能

通过DMA硬件模块，把源内存数据上的部分区域拷贝到目标内存上的部分区域。

• 语法

```
MI_S32 MI_SYS_BufBlitPa(MI_SYS_FrameData_t *pstDstBuf,
MI_SYS_WindowRect_t *pstDstRect, MI_SYS_FrameData_t
*pstSrcBuf, MI_SYS_WindowRect_t *pstSrcRect);
```

• 形参

参数名称	参数含义	输入/输出
pstDstBuf	目标内存物理首地址	输入
pstDstRect	目标内存拷贝的区域	输入
pstSrcBuf	源内存物理首地址	输入
pstSrcRect	源内存拷贝的区域	输入

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

• 注意

- pstDstRect/pstSrcRect数据范围是以pstDstBuf/pstSrcBuf所描述的首地址为（0，0）点开始，宽高都是以其中的ePixelFormat来计算每一个pixel所移动的内存数据大小来进行部分填充。
- pstDstBuf/pstSrcBuf中u16Width、u16Height、phyAddr、u32Stride、ePixelFormat为必填，其余数值无意义。
- 源内存或者目标内存的区域部分超过了其本来的范围，则只会拷贝其未超过部分上的数据。

• 举例

```

MI_S32 ret = MI_SUCCESS;
vdisp_copyinfo_plane_t *plane;
MI_SYS_FrameData_t stSrcFrame, stDstFrame;
MI_SYS_WindowRect_t stSrcRect, stDstRect;

plane = &copyinfo->plane[0];
stSrcFrame.ePixelFormat = E_MI_SYS_PIXEL_FRAME_I8;
stSrcFrame.phyAddr[0] = plane->src_paddr;
stSrcFrame.u16Width = plane->width;
stSrcFrame.u16Height = plane->height;
stSrcFrame.u32Stride[0] = plane->src_stride;

stDstFrame.ePixelFormat = E_MI_SYS_PIXEL_FRAME_I8;
stDstFrame.phyAddr[0] = plane->dst_paddr;
stDstFrame.u16Width = plane->width;
stDstFrame.u16Height = plane->height;
stDstFrame.u32Stride[0] = plane->dst_stride;

stSrcRect.u16X = 0;
stSrcRect.u16Y = 0;
stSrcRect.u16Height = stSrcFrame.u16Height;
stSrcRect.u16Width = stSrcFrame.u16Width;

stDstRect.u16X = 0;
stDstRect.u16Y = 0;
stDstRect.u16Height = stDstFrame.u16Height;
stDstRect.u16Width = stDstFrame.u16Width;

ret = MI_SYS_BufBlitPa(&stDstFrame, &stDstRect, &stSrcFrame,
&stSrcRect);

return ret;

```

- 相关主题

[MI_SYS_BufFillPa](#) [#2514-mi_sys_buffillpa]

2.5.16. MI_SYS_ConfigPrivateMMAPool

- 功能

为模块配置私有MMA Heap.

- 语法

```
MI_S32
MI_SYS_ConfigPrivateMMAPool(MI_SYS_GlobalPrivPoolConfig_t
*pstGlobalPrivPoolConf);
```

- 形参

参数名称	参数含义	输入/输出
pstGlobalPrivPoolConf	配置私有Mma Pool结构体指针	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

- 设备私有MMA Heap与通道私有MMA Heap不能共存。
- 建议在[MI_SYS_Init](#) [#231-mi_sys_init]后就为各模块创建好私有MMA heap。
- 当pstGlobalPrivPoolConf->bCreate 为TRUE时，创建私有POOL，为FALSE时，销毁私有POOL
- 各种eConfigType使用场景如下：

eConfigType	适用场景
E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL	为绑定为 E_MI_SYS_BIND_TYPE_HW_RIN模 式的vpe与venc端口设置私有ring heap pool
E_MI_SYS_PRE_CHN_PRIVATE_POOL	为模块通道设置私有heap pool
E_MI_SYS_PRE_DEV_PRIVATE_POOL	为模块设备设置私有heap pool
E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL	为模块output port设置heap pool · 设置后该port口的ouput buf优先从 其中分配

- 举例

假设场景为：IPC 下两路流，一路Main Stream H265最大分辨率1920*1080，一路Sub Stream H264最大分辨率720*576。

VENC 创建MMA Heap:

```

/Main Sream:
//为通道1创建大小为38745760的私有MMA heap
MI_SYS_GlobalPrivPoolConfig_t stConfig;
memset(&stConfig , 0 , sizeof(stConfig));
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule =
E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 1;
stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize =
38745760;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

//Sub Stream:
//为通道2创建大小为805152的私有MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule =
E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 2;

```

```

stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize =
805152;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

```

VENEC 销毁MMA Heap:

```

// Main Stream:
//销毁通道1的私有MMA heap
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule =
E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 1;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

//Sub Stream:
//销毁通道2的私有MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule =
E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 2;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

```

VPE 创建MMA Heap:

```

//为设备0创建大小为0x4f9200的私有MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_DEV_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreDevPrivPoolConfig.eModule =
E_MI_MODULE_ID_VPE;
stConfig.uConfig.stPreDevPrivPoolConfig.u32Devid = 0;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

```

VPE 销毁MMA Heap:

```

//销毁设备0的私有MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_DEV_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreDevPrivPoolConfig.eModule =
E_MI_MODULE_ID_VPE;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

```

创建VPE与VENC之间的ring MMA Heap：

```
stConfig.eConfigType = E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.
stPrevPe2VencRingPrivPoolConfig.u32VencInputRingPoolStaticSize =
8*1024*1024;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);
```

销毁VPE与VENC之间的ring MMA Heap：

```
stConfig.eConfigType = E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL;
stConfig.bCreate = FALSE;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);
```

不同分辨率或开启不同功能size大小都不一样，详细参数请根据使用的场景和规格计算。

- 相关主题

[MI_SYS_PrivateDevChnHeapAlloc](#) [#2517-mi_sys_privatedevchnheapalloc]

[MI_SYS_PrivateDevChnHeapFree](#) [#2518-mi_sys_privatedevchnheapfree]

2.5.17. MI_SYS_PrivateDevChnHeapAlloc

- 功能

从模块设备或通道私有MMA Pool 申请内存.

- 语法

```
MI_S32 MI_SYS_PrivateDevChnHeapAlloc(MI_ModuleId_e eModule,
MI_U32 u32Devid,MI_S32 s32ChnId, MI_U8 *pu8BufName, MI_U32
u32blkSize, MI_PHY *pphyAddr,MI_BOOL bTailAlloc);
```

- 形参

参数名称	参数含义	输入/输出
eModule	模块ID	输入
u32DevId	模块的设备ID	输入
s32ChnId	模块的通道ID	输入
pu8BufName	申请私有内存的名字，传Null时，使用"app-privAlloc"作为默认的名字	输入
u32blkSize	申请私有内存的大小	输入
pphyAddr	分配出的私有内存的物理地址	输出
bTailAlloc	是否从通道私有pool的尾部申请 0-否，1-是	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

当s32ChnId 等于 -1时，从模块设备私有MMA Pool 申请内存；

当s32ChnId 大于 -1时，从模块通道私有MMA Pool申请内存；

使用该接口时，如果没有配置相应私有池或者私有池中内存不足申请失败，会默认从mma_heap_name0中申请内存。

- 举例

MI_SYS_PrivateDevChnHeapAlloc调用Sample

```
MI_PHY *pphyAddr = NULL;
MI_SYS_GlobalPrivPoolConfig_t stConfig;
Memset(&stConfig, 0, sizeof(stConfig));
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
```



```

stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig. eModule =
E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig. u32DevId = 0;
stConfig.uConfig.stPreChnPrivPoolConfig. u32Channel= 1;
stConfig.uConfig.stPreChnPrivPoolConfig. u32PrivateHeapSize =
38745760;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

ret = MI_SYS_PrivateDevChnHeapAlloc(E_MI_MODULE_ID_VENC, 0, 1,
NULL, 4096, pphyAddr, FALSE);
if(ret != MI_SUCCESS)
{
    stConfig.bCreate = FALSE;
    MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);
    printf("alloc buff from chn private heap failed\n");
    return -1;
}

//do something...

MI_SYS_PrivateDevChnHeapFree(E_MI_MODULE_ID_VENC, 0, 1,
*pphyAddr);
stConfig.bCreate = FALSE;
MI_SYS_ConfigDevChnPrivateMMAHeap(&stConfig);

```

- 相关主题

[MI_SYS_ConfigPrivateMMAPool](#) [#2516-mi_sys_configprivatemmapool]

[MI_SYS_PrivateDevChnHeapFree](#) [#2518-mi_sys_privatedevchnheapfree]

2.5.18. MI_SYS_PrivateDevChnHeapFree

- 功能

从模块设备或通道私有MMA Pool 释放内存.

- 语法

```

MI_S32 MI_SYS_PrivateDevChnHeapFree(MI_ModuleId_e eModule,
MI_U32 u32DevId,MI_S32 s32ChnId, MI_PHY phyAddr);

```

- 形参

参数名称	参数含义	输入/输出
eModule	模块ID	输入
u32DevId	模块的设备ID	输入
s32ChnId	模块的通道ID	输入
phyAddr	需要释放的内存对应的物理地址	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

当s32ChnId 等于-1时，从模块设备私有MMA Pool释放内存；

当s32ChnId 大于-1时，从模块通道私有MMA Pool释放内存；

如果释放的物理地址不在私有池内存范围内或者没有配置对应的私有池，会去检查该地址是否从mma_heap_name0中分配出去的地址并释放。

- 举例

参见[MI_SYS_PrivateDevChnHeapAlloc调用Sample](#) [#jump5]举例。

- 相关主题

[MI_SYS_ConfigPrivateMMAPool](#) [#2516-mi_sys_configprivatemmapool]

[MI_SYS_PrivateDevChnHeapAlloc](#) [#2517-mi_sys_privatedevchnheapalloc]

2.5.19. MI_SYS_Va2Pa

- 功能

将MI内存块的CPU虚拟地址转成内存物理地址。

- 语法

```
MI_S32 MI_SYS_Va2Pa (void *pVirtualAddress, MI_PHY *pPhyAddr);
```

- 形参

参数名称	参数含义	输入/输出
pVirtualAddress	CPU虚拟地址指针	输入
pPhyAddr	存储内存块物理地址的指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

- 注意

使用该接口时，请确定输入的CPU虚拟地址有效且指向MI中分配的地址空间。

- 举例

MI_SYS_Va2Pa调用Sample

```
MI_PHY phySrcBufAddr = 0;
MI_PHY phyAddrVa2Pa = 0;
void *pVirSrcBufAddr = NULL;
MI_U32 srcBuffSize = 1920 * 1980 * 3 / 2;
srcBuffSize = ALIGN_UP(srcBuffSize, 4096);

ret = MI_SYS_MMA_Alloc(NULL, srcBuffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
    return -1;
}
```

```

    ret = MI_SYS_Mmap(phySrcBufAddr, srcBuffSize,
&pVirSrcBufAddr, TRUE);
    if(ret != MI_SUCCESS)
    {
        MI_SYS_MMA_Free(phySrcBufAddr);
        printf("mmap src buff failed\n");
        return -1;
    }

    MI_SYS_Va2Pa(pVirSrcBufAddr, &phyAddrVa2Pa);
    assert(phySrcBufAddr == phyAddrVa2Pa);
    memset(pVirSrcBufAddr, 0, srcBuffSize);
    MI_SYS_FlushInvCache(pVirSrcBufAddr,srcBuffSize);
    MI_SYS_Munmap(pVirSrcBufAddr, srcBuffSize);
    MI_SYS_MMA_Free(phySrcBufAddr);

```

- 相关主题

[MI_SYS_Mmap](#) [#259-mi_sys_mmap]

2.5.20. MI_SYS_GetMmaPoolInfo

- 功能

获取指定mma 池的总空闲大小和物理上连续的最大的内存长度。

- 语法

```

MI_S32 MI_SYS_GetMmaPoolInfo (MI_SYS_MmaPoolInfo_t
*pstMmaPoolInfo);

```

- 形参

参数名称	参数含义	输入/输出
bPrivatePool	pstMmaPoolInfo成员，是否获取私有池内存信息 TRUE：配置uConfig.stChnPort FALSE：配置u8MMAHeapName	输入
uConfig	pstMmaPoolInfo成员 uConfig.stChnPort：配置需要查询的私有池信息 uConfig.u8MMAHeapName：要查询的mma heap name	输入
u32TotalFreeSize	pstMmaPoolInfo成员，内存池总空闲大小	输出
u32MaxContiFreeSize	pstMmaPoolInfo成员，内存池物理连续的最大的内存长度	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_sys_datatype.h、mi_sys.h
 - 库文件：libmi_sys.a / libmi_sys.so
- 注意
 - 由于内存状态动态变化，该接口获取的瞬时值只代表这一时刻的内存状态。
 - bPrivatePool = TRUE时，配置stChnPort：

stChnPort.u32ChnId == -1 时，获取模块设备的私有池信息，对应E_MI_SYS_PRE_DEV_PRIVATE_POOL类型的私有池；

stChnPort.u32ChnId != -1 && stChnPort.u32PortId == -1 时，获取设备通道的私有池信息，对应E_MI_SYS_PRE_CHN_PRIVATE_POOL类型的私有池；

stChnPort.u32ChnId != -1 && stChnPort.u32PortId != -1 时，获取模块通道output port的私有池信息，对应E_MI_SYS_PRE_CHN_PORT_OUTPUT_POOL类型的私有池。

- 举例

MI_SYS_GetMmaPoolInfo调用Sample

```
// 获取模块设备的私有池内存信息
MI_SYS_MmaPoolInfo stMmaPoolInfo;
memset(&stMmaPoolInfo, 0, sizeof(stMmaPoolInfo));

stMmaPoolInfo.bPrivatePool = TRUE;
stMmaPoolInfo.uConfig.stChnPort.eModId = E_MI_MODULE_ID_VENC;
stMmaPoolInfo.uConfig.stChnPort.u32DevId=0;
stMmaPoolInfo.uConfig.stChnPort.u32ChnId=-1;
stMmaPoolInfo.uConfig.stChnPort.u32PortId=-1;
MI_SYS_GetMmaPoolInfo(&stMmaPoolInfo);
// 获取模块设备通道的私有池内存信息
MI_SYS_MmaPoolInfo stMmaPoolInfo;
memset(&stMmaPoolInfo, 0, sizeof(stMmaPoolInfo));

stMmaPoolInfo.bPrivatePool = TRUE;
stMmaPoolInfo.uConfig.stChnPort.eModId = E_MI_MODULE_ID_VENC;
stMmaPoolInfo.uConfig.stChnPort.u32DevId=0;
stMmaPoolInfo.uConfig.stChnPort.u32ChndId=0;
stMmaPoolInfo.uConfig.stChnPort.u32PortId=-1;
MI_SYS_GetMmaPoolInfo(&stMmaPoolInfo);

// 获取设备通道output port的私有池内存信息
MI_SYS_MmaPoolInfo stMmaPoolInfo;
memset(&stMmaPoolInfo, 0, sizeof(stMmaPoolInfo));

stMmaPoolInfo.bPrivatePool = TRUE;
stMmaPoolInfo.uConfig.stChnPort.eModId = E_MI_MODULE_ID_VENC;
stMmaPoolInfo.uConfig.stChnPort.u32DevId=0;
stMmaPoolInfo.uConfig.stChnPort.u32ChndId=1;
stMmaPoolInfo.uConfig.stChnPort.u32PortId=0;
MI_SYS_GetMmaPoolInfo(&stMmaPoolInfo);

// 获取mma heap内存池信息
MI_SYS_MmaPoolInfo stMmaPoolInfo;
memset(&stMmaPoolInfo, 0, sizeof(stMmaPoolInfo));

stMmaPoolInfo.bPrivatePool = FALSE;
memcpy(stMmaPoolInfo.uConfig.u8MMAHeapName, "mma_heap_name0", sizeof("mma_heap_name"));
MI_SYS_GetMmaPoolInfo(&stMmaPoolInfo);
```

- 相关主题

[MI_SYS_ConfigPrivateMMAPool](#) [#2516-mi_sys_configprivatemmapool]

3. 数据类型

3.1. 数据结构描述格式说明

本手册使用 5个参考域描述数据类型的相关信息，它们作用如 个参考域描述数据类型的相关信息，它们作用如 表 3-1所示。

表3-1 数据结构描述格式说明

标签	功能
说明	简要描述数据类型的主要功能
定义	列出数据类型的定义语句
成员	列出数据结构的成员及含义
注意事项	列出使用数据类型时应注意的事项
相关数据类型及接口	列出与本数据类型相关联的其他数据类型和接口

3.2. 数据结构列表

相关数据类型、数据结构定义如下：

数据类型	定义
MI_ModuleId_e [#321-mi_moduleid_e]	定义模块ID枚举类型
数据类型	定义
MI_ERROR_ID_E [#322-mi_error_id_e]	定义错误枚举类型

MI_SYS_PixelFormat_e [#322-mi_sys_pixelformat_e]	定义像素枚举类型
MI_SYS_CompressMode_e [#323-mi_sys_compressmode_e]	定义压缩方式枚举类型
MI_SYS_FrameTileMode_e [#324-mi_sys_frametilemode_e]	定义Tile格式枚举类型
MI_SYS_FieldType_e [#325-mi_sys_fieldtype_e]	定义Field枚举类型
MI_SYS_BufDataType_e [#326-mi_sys_bufdatatype_e]	定义模块ID枚举类型
MI_SYS_FrameIsPInfoType_e [#327-mi_sys_frameispinfotype_e]	定义frame data携带的ISP info枚举类型
MI_SYS_ChnPort_t [#328-mi_sys_chnport_t]	定义模块设备通道结构体
MI_SYS_MetaData_t [#329-mi_sys_metadata_t]	定义码流MetaData的结构体
MI_SYS_RawData_t [#3210-mi_sys_rawdata_t]	定义码流RawData的结构体
MI_SYS_WindowRect_t [#3211-mi_sys_windowrect_t]	定义Window坐标的结构体
MI_SYS_FrameData_t [#3212-mi_sys_framedata_t]	定义码流FrameData的结构体
MI_SYS_BufInfo_t [#3213-mi_sys_bufinfo_t]	Buf信息结构体
MI_SYS_FrameBufExtraConfig_t [#3214-mi_sys_framebufextraconfig_t]	定义码流Frame buffer额外配置的结构体
MI_SYS_BufFrameConfig_t [#3215-mi_sys_bufframeconfig_t]	Frame buf配置信息结构体
MI_SYS_BufRawConfig_t [#3216-mi_sys_bufrawconfig_t]	Raw buf配置信息结构体
MI_SYS_MetaDataConfig_t [#3217-mi_sys_metadataconfig_t]	Meta buf配置信息结构体
数据类型 BufConf_t [#3218-mi_sys_bufconf_t]	配置Buf信息结构体

MI_SYS_Version_t [#3219-mi_sys_version_t]	Sys版本信息结构体
MI_VB_PoolListConf_t [#3220-mi_vb_poollistconf_t]	描述VB Pool链表信息的结构体
MI_SYS_BindType_e [#3221-mi_sys_bindtype_e]	定义前后级工作模式的枚举类型
MI_SYS_FrameData_PhySignalType [#3222-mi_sys_framedata_physignaltpe]	描述frame data隶属的buffer类型的枚举类型
MI_SYS_InsidePrivatePoolType_e [#3223-mi_sys_insideprivatepooltype_e]	描述创建的私有MMA POOL类型的枚举类型
MI_SYS_PerChnPrivHeapConf_t [#3224-mi_sys_perchnprivheapconf_t]	定义描述通道私有MMA Pool的结构体
MI_SYS_PerDevPrivHeapConf_t [#3225-mi_sys_perdevprivheapconf_t]	定义描述设备私有MMA Pool的结构体
MI_SYS_PerVpe2VencRingPoolConf_t [#3226-mi_sys_pervpe2vencringpoolconf_t]	定义描述VPE与VENC之间的私有Ring MMA Pool的结构体
MI_SYS_PerChnPortOutputPool_t [#3227-mi_sys_perchnportoutputpool_t]	定义描述ouput port的私有MMA Pool的结构体
MI_SYS_GlobalPrivPoolConfig_t [#3228-mi_sys_globalprivpoolconfig_t]	定义描述配置私有 MMA Pool的结构体
MI_SYS_FrameIsplInfo_t [#3229-mi_sys_frameisplinfo_t]	定义frame data中ISP info的结构体
MI_SYS_InitParam_t [#3230-mi_sys_initparam_t]	定义Sys设备初始化参数
MI_SYS_BufFrameMultiPlaneConfig_t [#3231-mi_sys_buffframemultiplaneconfig_t]	MultiPlane Frame buf配置信息结构体
MI_SYS_FrameDataSubPlane_t [#3232-mi_sys_framedatasubplane_t]	SubPlane Frame buf配置信息结构体
MI_SYS_FrameDataMultiPlane_t [#3233-mi_sys_framedatamultiplane_t]	MultiPlane Frame buf配置信息结构体
数据类型	定义
MI_SYS_GlobalFlagType_e [#3234-	定义全局Flag类型

mi_sys_globalflagtype_e]

MI_SYS_GlobalFlagParam_t [#3235-
mi_sys_globalflagparam_t]

定义设置全局Flag参数

MI_SYS_MmaPoolInfo_t [#3236-
mi_sys_mmapoolinfo_t]

定义描述内存池的结构体

3.2.1. MI_ModuleId_e

- 说明
定义模块ID枚举类型。
- 定义

```
typedef enum
{
    E_MI_MODULE_ID_IVE      = 0,
    E_MI_MODULE_ID_VDF      = 1,
    E_MI_MODULE_ID_VENC      = 2,
    E_MI_MODULE_ID_RGN      = 3,
    E_MI_MODULE_ID_AI        = 4,
    E_MI_MODULE_ID_AO        = 5,
    E_MI_MODULE_ID_VIF       = 6,
    E_MI_MODULE_ID_VPE       = 7,
    E_MI_MODULE_ID_VDEC      = 8,
    E_MI_MODULE_ID_SYS       = 9,
    E_MI_MODULE_ID_FB        = 10,
    E_MI_MODULE_ID_HDMI      = 11,
```

```
E_MI_MODULE_ID_DIVP    = 12,

E_MI_MODULE_ID_GFX     = 13,

E_MI_MODULE_ID_VDISP   = 14,

E_MI_MODULE_ID_DISP    = 15,

E_MI_MODULE_ID_OS      = 16,

E_MI_MODULE_ID_IAE     = 17,

E_MI_MODULE_ID_MD      = 18,

E_MI_MODULE_ID_OD      = 19,

E_MI_MODULE_ID_SHADOW  = 20,

E_MI_MODULE_ID_WARP    = 21,

E_MI_MODULE_ID_UAC     = 22,

E_MI_MODULE_ID_LDC     = 23,

E_MI_MODULE_ID_SD      = 24,

E_MI_MODULE_ID_PANEL   = 25,

E_MI_MODULE_ID_CIPHER  = 26,

E_MI_MODULE_ID_SNR     = 27,

E_MI_MODULE_ID_WLAN    = 28,

E_MI_MODULE_ID_IPU     = 29,

E_MI_MODULE_ID_MIPITX  = 30,

E_MI_MODULE_ID_GYRO    = 31,

E_MI_MODULE_ID_JPD     = 32,

E_MI_MODULE_ID_PSPI    = 33,

E_MI_MODULE_ID_MAX,

} MI_ModuleId_e;
```

- 成员

模块ID	模块ID HEX 值	成员名称	模块
0	0x00	E_MI_MODULE_ID_IVE	图像智能算子IVE的 模块ID
1	0x01	E_MI_MODULE_ID_VDF	视频智能算法框架模块VDF的模块ID
2	0x02	E_MI_MODULE_ID_VENC	视频编码模块VPE的 模块ID
3	0x03	E_MI_MODULE_ID_RGN	OSD叠加和遮挡模块 REG的模块ID
4	0x04	E_MI_MODULE_ID_AI	音频输入模块AI的模 块ID
5	0x05	E_MI_MODULE_ID_AO	音频输出模块AO的 模块ID
6	0x06	E_MI_MODULE_ID_VIF	视频输入VIF的模块 ID
7	0x07	E_MI_MODULE_ID_VPE	视频图像处理模块 VPE的模块ID
8	0x08	E_MI_MODULE_ID_VDEC	视频处理VPE的模块 ID
9	0x09	E_MI_MODULE_ID_SYS	系统模块SYS的模块 ID
10	0x0A	E_MI_MODULE_ID_FB	SStar UI显示 FrameBuffer Device 模块的模块ID
11 模块ID	0x0B 模块ID HEX 值	E_MI_MODULE_ID_HDMI 成员名称	HMDI模块ID 模块
12	0x0C	E_MI_MODULE_ID_DIVP	视频DI及后处理模块

			DIVP的模块ID
13	0x0D	E_MI_MODULE_ID_GFX	2D图形处理加速模块GFX的模块ID
14	0x0E	E_MI_MODULE_ID_VDISP	图像拼图模块VDISP的模块ID
15	0x0F	E_MI_MODULE_ID_DISP	视频显示模块DISP的模块ID
16	0x10	E_MI_MODULE_ID_OS	RTOS系统模块ID
17	0x11	E_MI_MODULE_ID_IAE	音频智能算子IAE的模块ID
18	0x12	E_MI_MODULE_ID_MD	移动侦测模块MD的模块ID
19	0x13	E_MI_MODULE_ID_OD	遮挡侦测模块OD的模块ID
20	0x14	E_MI_MODULE_ID_SHADOW	虚拟MI框架SHADOW的模块ID
21	0x15	E_MI_MODULE_ID_WARP	图像畸形校正算法WARP的模块ID
22	0x16	E_MI_MODULE_ID_UAC	USB Aduio Class专用ALSA设备的模块ID
23	0x17	E_MI_MODULE_ID_LDC	图像畸形校正算法LDC的模块ID
24	0x18	E_MI_MODULE_ID_SD	图像缩放功能的模块ID
25	0x19	E_MI_MODULE_ID_PANEL	屏显示PANEL的模块ID
模块ID 26	模块ID HEX 0x1A	成员名称 E_MI_MODULE_ID_CIPHER	模块 芯片加密CIPHER的 模块ID

27	0x1B	E_MI_MODULE_ID_SNR	Sensor传感器模块ID
28	0x1C	E_MI_MODULE_ID_WLAN	无线通信网络WLAN的模块ID
29	0x1D	E_MI_MODULE_ID_IPU	图像识别处理IPU的模块ID
30	0x1E	E_MI_MODULE_ID_MIPITX	使用MIPI协议发送数据模块ID
31	0x1F	E_MI_MODULE_ID_GYRO	陀螺仪的模块ID
32	0x20	E_MI_MODULE_ID_JPD	JPD处理的模块ID
33	0x21	E_MI_MODULE_ID_PSPI	使用spi协议发送/接收数据的模块ID

3.2.2. MI_SYS_PixelFormat_e

- 说明
定义像素枚举类型。
- 定义

```
typedef enum
{
    E_MI_SYS_PIXEL_FRAME_YUV422_YUYV = 0,
    E_MI_SYS_PIXEL_FRAME_ARGB8888,
    E_MI_SYS_PIXEL_FRAME_ABGR8888,
    E_MI_SYS_PIXEL_FRAME_BGRA8888,
    E_MI_SYS_PIXEL_FRAME_RGB565,
    E_MI_SYS_PIXEL_FRAME_ARGB1555,
```

```

E_MI_SYS_PIXEL_FRAME_ARGB4444,

E_MI_SYS_PIXEL_FRAME_I2,

E_MI_SYS_PIXEL_FRAME_I4,

E_MI_SYS_PIXEL_FRAME_I8,

E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422,

E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420,

E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420_NV21,

E_MI_SYS_PIXEL_FRAME_YUV_TILE_420,

E_MI_SYS_PIXEL_FRAME_YUV422_UYVY,

E_MI_SYS_PIXEL_FRAME_YUV422_YVYU,

E_MI_SYS_PIXEL_FRAME_YUV422_VYUY,

E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR,

E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR,

E_MI_SYS_PIXEL_FRAME_FBC_420,

E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE,

E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM = E_MI_SYS_PIXEL_FRAME_RGB
1,

E_MI_SYS_PIXEL_FRAME_RGB888,

E_MI_SYS_PIXEL_FRAME_BGR888,

E_MI_SYS_PIXEL_FRAME_GRAY8,

E_MI_SYS_PIXEL_FRAME_FORMAT_MAX,

} MI_SYS_PixelFormat_e;

```

- 成员

成员名称	描述
------	----

E_MI_SYS_PIXEL_FRAME_YUV422_YUYV	YUV422_YUYV格式
E_MI_SYS_PIXEL_FRAME_ARGB8888	ARGB8888格式
E_MI_SYS_PIXEL_FRAME_ABGR8888	ABGR8888格式
E_MI_SYS_PIXEL_FRAME_BGRA8888	BGRA8888格式
E_MI_SYS_PIXEL_FRAME_RGB565	RGB565格式
E_MI_SYS_PIXEL_FRAME_ARGB1555	ARGB1555格式
E_MI_SYS_PIXEL_FRAME_ARGB4444	ARGB4444格式
E_MI_SYS_PIXEL_FRAME_I2	2 bpp格式
E_MI_SYS_PIXEL_FRAME_I4	4 bpp格式
E_MI_SYS_PIXEL_FRAME_I8	8 bpp格式
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422	YUV422 semi-planar格式
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420	YUV420sp nv12格式
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420_NV21	YUV420sp nv21格式
E_MI_SYS_PIXEL_FRAME_YUV_TILE_420	内部自定义TILE格式
E_MI_SYS_PIXEL_FRAME_YUV422_UYVY	YUV422_UYVY格式
E_MI_SYS_PIXEL_FRAME_YUV422_YVYU	YUV422_YVYU格式
E_MI_SYS_PIXEL_FRAME_YUV422_VYUY	YUV422_VYUY格式
E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR	YUV422_PLANAR格式
E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR	YUV420_PLANAR格式
E_MI_SYS_PIXEL_FRAME_FBC_420	内部自定义格式
成员名称 E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE	描述 RGB raw data 组合格式的起

	始值
E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM	RGB raw data 组合格式的最大值
E_MI_SYS_PIXEL_FRAME_RGB888	RGB888格式
E_MI_SYS_PIXEL_FRAME_BGR888	BGR888格式
E_MI_SYS_PIXEL_FRAME_GRAY8	8bit 灰度格式
E_MI_SYS_PIXEL_FRAME_FORMAT_MAX	枚举的最大值

- 注意事项

E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE, E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM 是提供给RGB raw data 组合格式使用的。此格式下，sensor提供的bit位ePixPrecision和排列顺序eBayerId, 通过RGB_BAYER_PIXEL宏对其组合生成RGB raw data 组合格式的enum值;

RGB_BAYER_PIXEL宏定义：#define RGB_BAYER_PIXEL(BitMode, PixelID) (E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE+ BitMode*E_MI_SYS_PIXEL_BAYERID_MAX+ PixelID)，其取值范围为: (E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE, E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM)

下面提供了一个使用的小例子，更详细的信息请参考对应模块的API文档。

```
static MI_SYS_PixelFormat_e getVpeRawType(STUB_SensorType_e
sensorType)
{
    MI_SYS_PixelFormat_e eVpeRawType =
E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM;

    MI_SNR_PlaneInfo_t stSnrPlane0Info;

    memset(&stSnrPlane0Info, 0x0, sizeof(MI_SNR_PlaneInfo_t));

    MI_SNR_GetPlaneInfo(E_MI_SNR_PAD_ID_0, 0,
&stSnrPlane0Info);
```

```

        eVpeRawType =
(MI_SYS_PixelFormat_e)RGB_BAYER_PIXEL(stSnrPlane0Info.ePixPrecisi
stSnrPlane0Info.eBayerId);

    return eVpeRawType;
}

```

3.2.3. MI_SYS_CompressMode_e

- 说明

定义压缩方式枚举类型。

- 定义

```

typedef enum
{

    E_MI_SYS_COMPRESS_MODE_NONE, //no compress

    E_MI_SYS_COMPRESS_MODE_SEG,
//compress unit is 256 bytes as a segment

    E_MI_SYS_COMPRESS_MODE_LINE,
//compress unit is the whole line

    E_MI_SYS_COMPRESS_MODE_FRAME,
//compress unit is the whole frame

    E_MI_SYS_COMPRESS_MODE_TO_8BIT,

    E_MI_SYS_COMPRESS_MODE_BUTT, //number

}MI_SYS_CompressMode_e;

```

- 成员

成员名称	描述
E_MI_SYS_COMPRESS_MODE_NONE	非压缩的视频格式
E_MI_SYS_COMPRESS_MODE_SEG	段压缩的视频格式
E_MI_SYS_COMPRESS_MODE_LINE	行压缩的视频格式，按照一行为一段进行压缩
E_MI_SYS_COMPRESS_MODE_FRAME	帧压缩的视频格式，将一帧数据进行压缩
E_MI_SYS_COMPRESS_MODE_TO_8BIT	Pixel 压缩到8bit
E_MI_SYS_COMPRESS_MODE_BUTT	视频压缩方式的数量

3.2.4. MI_SYS_FrameTileMode_e

- 说明
定义Tile格式枚举类型。
- 定义

```
typedef enum
{
    E_MI_SYS_FRAME_TILE_MODE_NONE = 0,

    E_MI_SYS_FRAME_TILE_MODE_16x16,      // tile mode 16x16
    E_MI_SYS_FRAME_TILE_MODE_16x32,      // tile mode 16x32
    E_MI_SYS_FRAME_TILE_MODE_32x16,      // tile mode 32x16
    E_MI_SYS_FRAME_TILE_MODE_32x32,      // tile mode 32x32
    E_MI_SYS_FRAME_TILE_MODE_MAX

} MI_SYS_FrameTileMode_e;
```

- 成员

成员名称	描述
E_MI_SYS_FRAME_TILE_MODE_NONE	None
E_MI_SYS_FRAME_TILE_MODE_16x16	16x16 mode
E_MI_SYS_FRAME_TILE_MODE_16x32	16x32 mode
E_MI_SYS_FRAME_TILE_MODE_32x16	32x16 mode
E_MI_SYS_FRAME_TILE_MODE_32x32	32x32 mode

3.2.5. MI_SYS_FieldType_e

- 说明
定义枚举类型。
- 定义

```
typedef enum
{
    E_MI_SYS_FIELDTYPE_NONE,          //< no field.
    E_MI_SYS_FIELDTYPE_TOP,           //< Top field only.
    E_MI_SYS_FIELDTYPE_BOTTOM,       //< Bottom field only.
    E_MI_SYS_FIELDTYPE_BOTH,         //< Both fields.
    E_MI_SYS_FIELDTYPE_NUM
} MI_SYS_FieldType_e;
```

- 成员

成员名称	描述
E_MI_SYS_FIELDTYPE_NONE	None
E_MI_SYS_FIELDTYPE_TOP	Top field only
E_MI_SYS_FIELDTYPE_BOTTOM	Bottom field only
E_MI_SYS_FIELDTYPE_BOTH	Both fields

3.2.6. MI_SYS_BufDataType_e

- 说明

定义模块ID枚举类型。

- 定义

```
typedef enum
{
    E_MI_SYS_BUFDATA_RAW = 0,
    E_MI_SYS_BUFDATA_FRAME,
    E_MI_SYS_BUFDATA_META,
    E_MI_SYS_BUFDATA_MULTIPANE,
} MI_SYS_BufDataType_e;
```

- 成员

成员名称	描述
E_MI_SYS_BUFDATA_RAW	Raw数据类型
E_MI_SYS_BUFDATA_FRAME	Frame数据类型
E_MI_SYS_BUFDATA_META	Meta数据类型
E_MI_SYS_BUFDATA_MULTIPLANE	MultiPlane数据类型

3.2.7. MI_SYS_FrameIsInfoType_e

- 说明
定义frame data携带的ISP info枚举类型。
- 定义

```
typedef enum
{
    E_MI_SYS_FRAME_ISP_INFO_TYPE_NONE,
    E_MI_SYS_FRAME_ISP_INFO_TYPE_GLOBAL_GRADIENT
}MI_SYS_FrameIsInfoType_e;
```

- 成员

成员名称	描述
E_MI_SYS_FRAME_ISP_INFO_TYPE_NONE	NONE
E_MI_SYS_FRAME_ISP_INFO_TYPE_GLOBAL_GRADIENT	ISP的全局梯度数据类型

3.2.8. MI_SYS_ChnPort_t

- 说明

定义模块设备通道结构体。

- 定义

```
typedef struct MI_SYS_ChnPort_s
{
    MI_ModuleId_e eModId;

    MI_U32 u32DevId;

    MI_U32 u32ChnId;

    MI_U32 u32PortId;

} MI_SYS_ChnPort_t;
```

- 成员

成员名称	描述
eModId	模块号
u32DevId	设备号
u32ChnId	通道号
u32PortId	端口号

3.2.9. MI_SYS_MetaData_t

- 说明

定义码流MetaData的结构体。

- 定义

```
typedef struct MI_SYS_MetaData_s
{
```

```

    void*   pVirAddr;
    MI_PHY  phyAddr;//notice that this is miu bus addr,not cpu
    bus addr.

    MI_U32  u32Size;
    MI_U32  u32ExtraData;    /*driver special flag*/
    MI_ModuleId_e eDataFromModule;
} MI_SYS_MetaData_t;

```

- 成员

成员名称	描述
pVirAddr	数据存放虚拟地址。
phyAddr	数据存放物理地址
u32Size	数据大小
u32ExtraData	driver special flag
eDataFromModule	来自哪个模块的数据

3.2.10. MI_SYS_RawData_t

- 说明

定义码流RawData的结构体。

- 定义

```

typedef struct MI_SYS_RawData_s
{
    void*   pVirAddr;
    MI_PHY  phyAddr;//notice that this is miu bus addr,not cpu
    bus addr.
    MI_U32  u32BufSize;

    MI_U32  u32ContentSize;
    MI_BOOL bEndOfFrame;
    MI_U64  u64SeqNum;
} MI_SYS_RawData_t;

```


- 成员

成员名称	描述
pVirAddr	码流包的地址。
phyAddr	码流包的物理地址
u32BufSize	Buf大小
u32ContentSize	数据实际占用buf大小
bEndOfFrame	当前帧是否结束。（预留参数）当前只支持帧模式下按帧传送。
u64SeqNum	当前帧的帧序号

- 注意事项

- 当前只支持按帧传送数据，每次需完整传送一帧数据。
- 码流帧数据附带PTS时，解码后输出数据输出相同PTS。当PTS=-1时，不参考系统时钟输出数据帧。

3.2.11. MI_SYS_WindowRect_t

- 说明

定义window坐标的结构体。

- 定义

```
typedef struct MI_SYS_WindowRect_s
{
    MI_U16 u16X;

    MI_U16 u16Y;

    MI_U16 u16Width;
```

```
MI_U16 u16Height;  
  
}MI_SYS_WindowRect_t;
```

- 成员

成员名称	描述
u16X	window起始位置的水平方向的值。
u16Y	window起始位置的垂直方向的值。
u16Width	window宽度。
u16Height	window高度。

3.2.12. MI_SYS_FrameData_t

- 说明

定义码流FrameData的结构体。

- 定义

```
//N.B. in MI_SYS_FrameData_t should never support u32Size,  
//for other values are enough, and not support u32Size is  
general standard method.  
typedef struct MI_SYS_FrameData_s  
{  
    MI_SYS_FrameTileMode_e eTileMode;  
    MI_SYS_PixelFormat_e ePixelFormat;  
    MI_SYS_CompressMode_e eCompressMode;  
    MI_SYS_FrameScanMode_e eFrameScanMode;  
    MI_SYS_FieldType_e eFieldType;  
    MI_SYS_FrameData_PhySignalType ePhylayoutType;  
  
    MI_U16 u16Width;  
    MI_U16 u16Height;  
    //in case ePhylayoutType equal to REALTIME_FRAME_DATA, pVirAddr  
    would be MI_SYS_REALTIME_MAGIC_PADDR and phyAddr would be  
    MI_SYS_REALTIME_MAGIC_VADDR
```

```

    void* pVirAddr[3];
    MI_PHY phyAddr[3]; //notice that this is miu bus addr, not
    cpu bus addr.
    MI_U32 u32Stride[3];
    MI_U32 u32BufSize; //total size that allocated for this
    buffer, include consider alignment.

    MI_U16 u16RingBufStartLine; //Valid in case
    RINGBUF_FRAME_DATA, u16RingBufStartLine must be LGE than 0 and
    less than u16Height
    MI_U16 u16RingBufRealTotalHeight; ///Valid in case
    RINGBUF_FRAME_DATA, u16RingBufStartLine must be LGE than
    u16Height

    MI_SYS_FrameIsInfo_t stFrameIsInfo; //isp info of each
    frame
    MI_SYS_WindowRect_t stContentCropWindow;
} MI_SYS_FrameData_t;

```

- 成员

成员名称	描述
eTileMode	Tile模式
ePixelFormat	像素格式
eCompressMode	压缩格式
eFrameScanMode	Frame scan模块
eFieldType	File类型
ePhylayoutType	frame data隶属的buffer的类型
u16Width	Frame宽度
u16Height	Frame高度
pVirAddr	虚拟地址
phyAddr	物理地址
u32Stride	图片每行所占字节数
u32BufSize	Sys分配给当前frame的实际buf大小
u16RingBufStartLine	ring mode时，frame data开始于ring buffer的行数
u16RingBufRealTotalHeight	ring mode时，frame data的真实高度
stFramelsplInfo	ISP info struct
stContentCropWindow	Crop info struct

3.2.13. MI_SYS_BufInfo_t

- 说明

定义码流信息的结构体。

- 定义

```
typedef struct MI_SYS_BufInfo_s
{
    MI_U64 u64Pts;
    MI_U64 u64SidebandMsg;
    MI_SYS_BufDataType_e eBufType;
    MI_U32 bEndOfStream : 1;
    MI_U32 bUsrBuf : 1;
    MI_U32 bDrop : 1;
    MI_U32 u32IrFlag : 2; // For Window Hello Usage: 0x00/off,
    0x01/on, 0x02/invalid
    MI_U32 u32Reserved : 27;
    MI_U32 u32SequenceNumber;    union
    {
        MI_SYS_FrameData_t stFrameData;
        MI_SYS_RawData_t stRawData;
        MI_SYS_MetaData_t stMetaData;
        MI_SYS_FrameDataMultiPlane_t stFrameDataMultiPlane;
    };
} MI_SYS_BufInfo_t;
```

- 成员

成员名称	描述
eBufType	Buf类型
u64Pts	时间戳
bEndOfStream	是否已发完所有信息
u32IrFlag	Ir标志位

3.2.14. MI_SYS_FrameBufExtraConfig_t

- 说明

定义码流Frame buffer额外配置的结构体。

- 定义

```
typedef struct MI_SYS_FrameBufExtraConfig_s
{
    //Buf alignment requirement in horizontal
    MI_U16 u16BufHAlignment;
    //Buf alignment requirement in vertical
    MI_U16 u16BufVAlignment;
    //Buf alignment requirement in chroma
    MI_U16 u16BufChromaAlignment;
    //Clear padding flag
    MI_BOOL bClearPadding;
}MI_SYS_FrameBufExtraConfig_t;
```

- 成员

成员名称	描述
u16BufHAlignment	水平方向对齐值
u16BufVAlignment	垂直方向对齐值
u16BufChromaAlignment	色度buffer size对齐值
bClearPadding	是否对buffer的边缘进行涂黑

3.2.15. MI_SYS_BufFrameConfig_t

- 说明

定义码流Frame buffer配置的结构体。

- 定义

```
typedef struct MI_SYS_BufFrameConfig_s
{
    MI_U16 u16Width;
    MI_U16 u16Height;
    MI_SYS_FrameScanMode_e eFrameScanMode;//
    MI_SYS_PixelFormat_e eFormat;
    MI_SYS_CompressMode_e eCompressMode;
}MI_SYS_BufFrameConfig_t;
```

- 成员

成员名称	描述
u16Width	Frame宽度
u16Height	Frame高度
eFrameScanMode	Frame Scan Mode
eFormat	Frame像素格式
eCompressMode Frame	压缩模式

3.2.16. MI_SYS_BufRawConfig_t

- 说明

定义码流raw buffer配置的结构体。

- 定义

```
typedef struct MI_SYS_BufRawConfig_s
{
    MI_U32 u32Size;
}MI_SYS_BufRawConfig_t;
```

- 成员

成员名称	描述
u32Size	Buf大小

3.2.17. MI_SYS_MetaDataConfig_t

- 说明

定义码流meta buffer配置的结构体。

- 定义

```
typedef struct MI_SYS_MetaDataConfig_s
{
    MI_U32 u32Size;
}MI_SYS_MetaDataConfig_t;
```

- 成员

成员名称	描述
u32Size	Buf大小

3.2.18. MI_SYS_BufConf_t

- 说明

定义码流Port Buf配置的结构体。

- 定义

```
typedef struct MI_SYS_BufConf_s
{
    MI_SYS_BufDataType_e eBufType;
    MI_U32 u32Flags;    //0 or MI_SYS_MAP_VA
    MI_U64 u64TargetPts;
    union
    {
        MI_SYS_BufFrameConfig_t stFrameCfg;
        MI_SYS_BufRawConfig_t stRawCfg;
        MI_SYS_MetaDataConfig_t stMetaCfg;
        MI_SYS_BufFrameMultiPlaneConfig_t stMultiPlaneCfg;
    };
}MI_SYS_BufConf_t;
```


- 成员

成员名称	描述
eBufType	Buf type
u32Flags	Buf是否map kernel space va

3.2.19. MI_SYS_Version_t

- 说明

定义Sys版本信息结构体。

- 定义

```
typedef struct MI_SYS_Version_s
{
    MI_U8 u8Version[128];
}MI_SYS_Version_t;
```

- 成员

成员名称	描述
u8Version[128]	描述sys版本信息的字符串buf

3.2.20. MI_VB_PoolListConf_t

- 说明

定义描述VB Pool链表信息的结构体。

- 定义

```
typedef struct MI_VB_PoolListConf_s
{
    MI_U32 u32PoolListCnt;

    MI_VB_PoolConf_t stPoolConf[MI_VB_POOL_LIST_MAX_CNT];
} MI_VB_PoolListConf_t;
```

- 成员

成员名称	描述
u32PoolListCnt	VB pool 链表成员数量
stPoolConf	VB pool 链表成员配置信息

3.2.21. MI_SYS_BindType_e

- 说明

定义前后级工作模式。

- 定义

```
typedef enum
{
    E_MI_SYS_BIND_TYPE_FRAME_BASE = 0x00000001,
    E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY = 0x00000002,
    E_MI_SYS_BIND_TYPE_REALTIME = 0x00000004,
    E_MI_SYS_BIND_TYPE_HW_AUTOSYNC = 0x00000008,
    E_MI_SYS_BIND_TYPE_HW_RING = 0x00000010
} MI_SYS_BindType_e;
```

- 成员

成员名称	描述
E_MI_SYS_BIND_TYPE_FRAME_BASE	frame mode，默认工作方式
E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY	低时延工作方式
E_MI_SYS_BIND_TYPE_REALTIME	硬件直连工作方式
E_MI_SYS_BIND_TYPE_HW_AUTOSYNC	前后级handshake，buffer size与图像分辨率一致
E_MI_SYS_BIND_TYPE_HW_RING	前后级handshake，ring buffer depth可以调小于图像分辨率高

- 注意事项

- 旧版本MI SYS不提供此功能，如没有找到，不用设置。
- FRAME_BASE和HW_RING类型使用的内存为DRAM，REALTIME类型使用的内存为SRAM。
- 由于底层限制，被HW_RING或REALTIME类型bind的input port不能使用MI_SYS_ChnInputPortGetBuf/ MI_SYS_ChnInputPortPutBuf接口，output port不能使用MI_SYS_ChnOutputPortGetBuf/ MI_SYS_ChnOutputPortPutBuf接口。
- 不同bindtype使用内存大小比较：FRAME_BASE类型 > HW_RING类型 > REALTIME类型。

3.2.22. MI_SYS_FrameData_PhySignalType

- 说明

描述frame data隶属的buffer类型的枚举类型。

- 定义

```
typedef enum
```

```

{

    REALTIME_FRAME_DATA,

    RINGBUF_FRAME_DATA,

    NORMAL_FRAME_DATA,

}MI_SYS_FrameData_PhySignalType;

```

- 成员

成员名称	描述
REALTIME_FRAME_DATA	以E_MI_SYS_BIND_TYPE_REALTIME模式 产生的frame data
RINGBUF_FRAME_DATA	以E_MI_SYS_BIND_TYPE_HW_RING模式 产生的frame data
NORMAL_FRAME_DATA	以E_MI_SYS_BIND_TYPE_FRAME_BASE模式产生的frame data

- 注意事项

旧版本MI SYS不提供此功能，如没有找到，不用设置。

3.2.23. MI_SYS_InsidePrivatePoolType_e

- 说明

描述创建的私有MMA POOL类型的枚举类型。

- 定义

```

typedef enum
{

    E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL = 0,

    E_MI_SYS_PER_CHN_PRIVATE_POOL=1,

    E_MI_SYS_PER_DEV_PRIVATE_POOL=2,

}

```

```
E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL=3,  
  
}MI_SYS_InsidePrivatePoolType_e;
```

- 成员

成员名称	描述
E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL	VPE与VENC 的私有Ring Pool 类型
E_MI_SYS_PER_CHN_PRIVATE_POOL	通道私有Pool 类型
E_MI_SYS_PER_DEV_PRIVATE_POOL	设备私有Pool 类型
E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL	输出端口私有Pool类型

- 注意事项

旧版本MI SYS不提供此功能，如没有找到，不用设置。

3.2.24. MI_SYS_PerChnPrivHeapConf_t

- 说明

定义描述通道私有MMA Pool的结构体。

- 定义

```
typedef struct MI_PerChnPrivHeapConf_s  
{  
  
    MI_ModuleId_e eModule;  
  
    MI_U32 u32DevId;  
  
    MI_U32 u32Channel;  
  
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];  
  
    MI_U32 u32PrivateHeapSize;
```

```
}MI_SYS_PerChnPrivHeapConf_t;
```

- 成员

成员名称	描述
eModule	模块ID
u32Devid	设备ID
u32Channel	通道ID
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	私有pool 大小

3.2.25. MI_SYS_PerDevPrivHeapConf_t

- 说明

定义描述设备私有MMA Pool的结构体。

- 定义

```
typedef struct MI_PerDevPrivHeapConf_s
{
    MI_ModuleId_e eModule;

    MI_U32 u32Devid;

    MI_U32 u32Reserve;

    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];

    MI_U32 u32PrivateHeapSize;
}MI_SYS_PerDevPrivHeapConf_t;
```

- 成员

成员名称	描述
eModule	模块ID
u32Devid	设备ID
u32Reserve	预留
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	私有pool 大小

3.2.26. MI_SYS_PerVpe2VencRingPoolConf_t

- 说明

定义描述VPE与VENC之间的私有Ring MMA Pool的结构体。

- 定义

```
typedef struct MI_SYS_PerVpe2VencRingPoolConf_s
{
    MI_U32 u32VencInputRingPoolStaticSize;

    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
}MI_SYS_PerVpe2VencRingPoolConf_t;
```

- 成员

成员名称	描述
u32VencInputRingPoolStaticSize	私有pool大小
u8MMAHeapName	Mma heap name

3.2.27. MI_SYS_PerChnPortOutputPool_t

- 说明
定义描述ouput port的私有MMA Pool的结构体。
- 定义

```
typedef struct MI_SYS_PerChnPortOutputPool_s
{
    MI_ModuleId_e eModule;

    MI_U32 u32Devid;

    MI_U32 u32Channel;

    MI_U32 u32Port;

    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];

    MI_U32 u32PrivateHeapSize;
}MI_SYS_PerChnPortOutputPool_t;
```

- 成员

成员名称	描述
eModule	模块ID
u32Devid	设备ID
u32Channel	通道ID
u32Port	端口ID
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	私有pool 大小

3.2.28. MI_SYS_GlobalPrivPoolConfig_t

- 说明

定义描述配置私有 MMA Pool的结构体。

- 定义

```
typedef struct MI_SYS_GlobalPrivPoolConfig_s
{
    MI_SYS_InsidePrivatePoolType_e eConfigType;

    MI_BOOL bCreate;

    union
    {
        MI_SYS_PerChnPrivHeapConf_t stPreChnPrivPoolConfig;

        MI_SYS_PerDevPrivHeapConf_t stPreDevPrivPoolConfig;

        MI_SYS_PerVpe2VencRingPoolConf_t stPreVpe2VencRingPrivPool;

        MI_SYS_PerChnPortOutputPool_t stPreChnPortOutputPrivPool;

    }uConfig;
}MI_SYS_GlobalPrivPoolConfig_t;
```

- 成员

成员名称	描述
eConfigType	私有pool类型
bCreate	是否创建私有pool。true:创建 false:销毁
uConfig	不同类型私有pool的结构体

3.2.29. MI_SYS_FrameIsplInfo_t

- 说明

定义frame data中ISP info的结构体。

- 定义

```
typedef struct MI_SYS_FrameIsplInfo_s
{
    MI_SYS_FrameIsplInfoType_e eType;

    union
    {
        MI_U32 u32GlobalGradient;

    }uIsplInfo;
}MI_SYS_FrameIsplInfo_t;
```

- 成员

成员名称	描述
eType	ISP info type
uIsplInfo	ISP info union

3.2.30. MI_SYS_InitParam_t

- 说明

SYS设备初始化参数。

- 定义

```
typedef struct MI_SYS_InitParam_s
{
    MI_U32 u32DevId;
    MI_U8 *u8Data;
} MI_SYS_InitParam_t;
```

- 成员

成员名称	描述
u32DevId	设备ID
u8Data	数据指针buffer

- 相关数据类型及接口

[MI_SYS_InitDev](#) [#2311-mi_sys_initdev]

3.2.31. MI_SYS_BufFrameMultiPlaneConfig_t

- 说明

定义码流multiplane buffer配置的结构体。

- 定义

```
typedef struct MI_SYS_BufFrameMultiPlaneConfig_s
{
    MI_U8 u8SubPlaneNum;

    MI_SYS_BufFrameConfig_t stFrameCfg;
}MI_SYS_BufFrameMultiPlaneConfig_t;
```

- 成员

成员名称	描述
-----	-----

|u8SubPlaneNum|Buf 个数|
|stFrameCfg|定义码流Frame buffer配置的结构体|

3.2.32. MI_SYS_FrameDataSubPlane_t

- 说明

定义码流SubPlane FrameData的结构体。

- 定义

```
typedef struct MI_SYS_FrameDataSubPlane_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;
    MI_U64 u64FrameId;

    MI_U16 u16Width;
    MI_U16 u16Height;
    void* pVirAddr[2];
    MI_PHY phyAddr[2];
    MI_U16 u16Stride[2];
    MI_U32 u32BufSize;
} MI_SYS_FrameDataSubPlane_t;
```

- 成员

成员名称	描述
ePixelFormat	像素格式
eCompressMode	压缩格式
u64FrameId	Frame id
u16Width	Frame宽度
u16Height	Frame高度
pVirAddr	虚拟地址
phyAddr	物理地址
U16Stride	图片每行所占字节数
u32BufSize	Sys分配给当前frame的实际buf大小

3.2.33. MI_SYS_FrameDataMultiPlane_t

- 说明

定义码流MultiPlane FrameData的结构体。

- 定义

```
typedef struct MI_SYS_FrameDataMultiPlane_s
{
    MI_U8 u8SubPlaneNum;
    MI_SYS_FrameDataSubPlane_t
    stSubPlanes[MI_SYS_MAX_SUB_PLANE_CNT];
} MI_SYS_FrameDataMultiPlane_t;
```

- 成员

成员名称	描述
u8SubPlaneNum	Buf个数
stSubPlanes	定义码流MultiPlane FrameData的结构体

3.2.34. MI_SYS_GlobalFlagType_e

- 说明 定义全局Flag类型。
- 定义

```
typedef enum
{
    E_MI_SYS_GLOBALFLAG_IR,
    E_MI_SYS_GLOBALFLAG_NUM,
} MI_SYS_GlobalFlagType_e;
```

- 成员

成员名称	描述
E_MI_SYS_GLOBALFLAG_IR	IR全局Flag

3.2.35. MI_SYS_GlobalFlagParam_t

- 说明
定义设置全局Flag参数。
- 定义

```
typedef struct MI_SYS_GlobalFlagParam_s
{
    MI_SYS_GlobalFlagType_e eFlagType;
    MI_BOOL bIsr;
    MI_U32 u32Value;
} MI_SYS_GlobalFlagParam_t;
```

- 成员

成员名称	描述
eFlagType	设置Flag的类型，目前只支持设置为MI_SYS_GLOBALFLAG_IR
blsr	该Flag设置是否会在中断中执行，若在中断中调用，需设置为1
u32Value	设置的全局标志位值，目前只支持设置IR标志位，0x01表示IR灯打开，0x00表示IR灯关闭

- 注意事项

设置IR全局Flag时，u32Value为0时则表示OFF，为1时则表示ON。

3.2.36. MI_SYS_MmaPoolInfo_t

- 说明

定义描述内存池的结构体。

- 定义

```
typedef struct MI_SYS_MmaPoolInfo_s
{
    MI_BOOL bPrivatePool;
    Union{
        MI_SYS_ChnPort_t stChnPort;
        MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
    }uConfig;
    MI_U32 u32TotalFreeSize;
    MI_U32 u32MaxContiFreeSize;
} MI_SYS_MmaPoolInfo_t;
```

- 成员

成员名称	描述
bPrivatePool	是否获取私有pool信息 TRUE：配置uConfig. stChnPort FALSE：配置u8MMAHeapName
uConfig	stChnPort：配置需要查询的私有Pool u8MMAHeapName：要查询的mma heap name
u32TotalFreeSize	内存池总空闲大小
u32MaxContiFreeSize	内存池物理连续的最大内存长度

4. 错误码

4.1. 错误码的组成

MI 返回的错误码为4字节，共32bits，由表4-1所示五部分组成：

表4-1 错误码的组成

位置	所占位数	描述
BIT [0-11]	12bits	错误类型，表示该错误码的具体错误含义。
BIT [12-15]	4bits	错误等级，固定返回2。
BIT [16-23]	8bits	模块ID，表示该错误码是属于哪个模块。
BIT [24-31]	8bits	固定为0xA0。

Tips：

我们可以简单的把错误码看成两个双字节（16bits）做快速解读，以错误码 0xA0092001 为例：

“A009”：表示该错误发生在模块ID为9的模块，通过查看“MI_ModuleId_e [#321-mi_moduleid_e]”的定义可知为MI_SYS模块。

“2001”：表示该错误的错误类型是1，即“设备 ID 超出合法范围”。

4.2. MI_SYS API错误码表

MI_SYS模块所有 API 返回的值都为4字节，0表示执行成功，其它值表示执行失败，需要注意的是，所有非0的返回值都应该是表4-2中列出的，否则为非法值。

表4-1 MI_SYS模块 API错误码

错误码	宏定义	描述
0xA0092001	MI_ERR_SYS_INVALID_DEVID	设备 ID 超出合法范围
0xA0092002	MI_ERR_SYS_INVALID_CHNID	通道组号错误或无效区域句柄
0xA0092003	MI_ERR_SYS_ILLEGAL_PARAM	参数超出合法范围
0xA0092004	MI_ERR_SYS_EXIST	重复创建已存在的设备、通道或资源
0xA0092005	MI_ERR_SYS_UNEXIST	试图使用或者销毁不存在的设备、通道或者资源
0xA0092006	MI_ERR_SYS_NULL_PTR	函数参数中有空指针
0xA0092007	MI_ERR_SYS_NOT_CONFIG	模块没有配置
0xA0092008	MI_ERR_SYS_NOT_SUPPORT	不支持的参数或者功能
0xA0092009	MI_ERR_SYS_NOT_PERM	该操作不允许，如试图修改静态配置参数
0xA009200C	MI_ERR_SYS_NOMEM	分配内存失败，如系统内存不足
错误码	宏定义	描述
0xA009200D	MI_ERR_SYS_NOBUF	分配缓存失败，如申请的数据缓冲区

0xA009200D	MI_ERR_SYS_BUF_OVERFLOW	缓冲区中数据太大
0xA009200E	MI_ERR_SYS_BUF_EMPTY	缓冲区中无数据
0xA009200F	MI_ERR_SYS_BUF_FULL	缓冲区中数据满
0xA0092010	MI_ERR_SYS_NOTREADY	系统没有初始化或没有加载相应模块
0xA0092011	MI_ERR_SYS_BADADDR	地址非法
0xA0092012	MI_ERR_SYS_BUSY	系统忙
0xA0092013	MI_ERR_SYS_CHN_NOT_STARTED	通道没有开始
0xA0092014	MI_ERR_SYS_CHN_NOT_STOPED	通道没有停止
0xA0092015	MI_ERR_SYS_NOT_INIT	模块没有初始化
0xA0092016	MI_ERR_SYS_INITED	模块已经初始化
0xA0092017	MI_ERR_SYS_NOT_ENABLE	通道或端口没有ENABLE
0xA0092018	MI_ERR_SYS_NOT_DISABLE	通道或端口没有DISABLE
0xA0092019	MI_ERR_SYS_TIMEOUT	超时
0xA009201A	MI_ERR_SYS_DEV_NOT_STARTED	设备没有开始
0xA009201B	MI_ERR_SYS_DEV_NOT_STOPED	设备没有停止
0xA009201C	MI_ERR_SYS_CHN_NO_CONTENT	通道没有资料
0xA009201D	MI_ERR_SYS_NOVASAPCE	映射虚拟地址失败
0xA009201E	MI_ERR_SYS_NOITEM	RingPool中没有record记录
0xA009201F	MI_ERR_SYS_FAILED	未明确定义的错误

5. PROCFS介绍

5.1. common

5.1.1. dump_mmap

- 调试信息

```
# ./config/dump_mmap
```

```
// # ./config/dump_mmap
u32TotalSize:0x10000000
u32Miu0Size:0x10000000
u32Miu1Size:0x0
u32MiuBoundary:0x10000000
u32MmapItemsNum:14
bIs4kAlign:1
bMiu1Enable:0
E_MMAP_ID_DUMMY2:GID=0,Addr=0x0,Size=0x200000,Layer=0,Align=0x0,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_CMDQ:GID=1,Addr=0x0,Size=0x30000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_GFX:GID=2,Addr=0x30000,Size=0x40000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_GE_VQ:GID=3,Addr=0x70000,Size=0x20000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_AO:GID=4,Addr=0x90000,Size=0x4b000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_AI:GID=5,Addr=0xdb000,Size=0x4b000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_XC_MLOAD:GID=6,Addr=0x126000,Size=0x40000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_HW_CURSOR:GID=7,Addr=0x166000,Size=0x20000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_VDEC_CPU:GID=8,Addr=0x200000,Size=0xa00000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_LX_MEM:GID=9,Addr=0xc00000,Size=0xe300000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MAC_MEM:GID=10,Addr=0xf00000,Size=0x100000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_XC_MAIN_FB:GID=11,Addr=0xf000000,Size=0x800000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_FB:GID=12,Addr=0xf800000,Size=0x400000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_MAD_R2:GID=13,Addr=0xfc00000,Size=0x400000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
```

- 调试信息分析

Common下存放的是配置API，目的如下：

- a. 把iniparser放在用户态做，iniparser的任何开销不带入内核。
- b. 方便调试时确认配置文件的实际内容，因为在parser时打印的内容可能会与实际设置的内容有偏差。

为了达到上面两个目的，导出了一些文件。

如下是导入config的配置文件：

```
MI_COMMON_AddDebugRawFile("config_info", /*(sizeof
MI_COMMON_AddDebugRawFile("pq_info", /*_pqBaseSize
MI_COMMON_AddDebugRawFile("noise_table", /*4096, *
MI_COMMON_AddDebugRawFile("motion_table", /*4096, *
MI_COMMON_AddDebugRawFile("motion_hdmi_dtv_table",
MI_COMMON_AddDebugRawFile("motion_comp_pc_table",
MI_COMMON_AddDebugRawFile("misc_table", /*4096, */
MI_COMMON_AddDebugRawFile("misc_luma_table", /*409
MI_COMMON_AddDebugRawFile("misc_param_table", /*40
```

将 `Systeminfo.c` 内parse的struct写入config_info。

将 `Pqloader.c` 内parse的struct写入pq_info和*_table，此处的table是连续的内存摆放，MISDK是一个间接数组指向一列一维数组。

如下是导入mmap的配置文件：

```
MI_COMMON_AddDebugRawFile("mmap_info",  
MI_COMMON_AddDebugFile("memory_info",
```

将 `Mmapinfo.c` 内parse的struct写入mmap_info和memory_info。因为这里比较简单，所以从mmap_info和memory_info读出来是格式化的可读信息。

5.1.2. dump_config

- 调试信息

```
# ./config/dump_config  
  
panel cnt:59  
PanelName:RM68200_720x1280 Intftype:11 timing:55  
PanelName:SAT070AT50H18BH_1024x600 Intftype:0 timing:55  
PanelName:BT1120OUTPUT_720p_60HZ Intftype:14 timing:55  
PanelName:BT6560OUTPUT_576p_50HZ Intftype:12 timing:55  
PanelName:ST7789V Intftype:16 timing:55  
PanelName:ILI9881C_800x1280 Intftype:11 timing:55  
PanelName:ILI9163C_128x128 Intftype:15 timing:55  
PanelName:DACOUT_576I Intftype:7 timing:4  
PanelName:DACOUT_480I Intftype:7 timing:2  
PanelName:DACOUT_1080P_24 Intftype:8 timing:10  
PanelName:DACOUT_1080P_25 Intftype:8 timing:11  
PanelName:DACOUT_1080P_30 Intftype:8 timing:12  
PanelName:DACOUT_1080P_50 Intftype:8 timing:15  
PanelName:DACOUT_720P_2997 Intftype:8 timing:6  
PanelName:DACOUT_720P_50 Intftype:8 timing:7  
PanelName:DACOUT_720P_5994 Intftype:8 timing:8  
PanelName:DACOUT_720P_60 Intftype:8 timing:9  
PanelName:DACOUT_1080P_2997 Intftype:8 timing:13  
PanelName:DACOUT_1080I_50 Intftype:7 timing:14  
PanelName:DACOUT_1080P_5994 Intftype:8 timing:16  
PanelName:DACOUT_1080I_60 Intftype:7 timing:17  
PanelName:DACOUT_1080P_60 Intftype:8 timing:18  
PanelName:DACOUT_576P Intftype:8 timing:5  
PanelName:DACOUT_480P Intftype:8 timing:3  
PanelName:DACOUT_2K2KP_30 Intftype:8 timing:27  
PanelName:DACOUT_2K2KP_60 Intftype:8 timing:28  
PanelName:DACOUT_4K2KP_24 Intftype:8 timing:33  
PanelName:DACOUT_4K2KP_25 Intftype:8 timing:34  
PanelName:DACOUT_4K2KP_30 Intftype:8 timing:35  
PanelName:DACOUT_4K2KP_50 Intftype:8 timing:36
```

```

PanelName:DACOUT_4K2KP_60 Intftype:8 timing:37
PanelName:DACOUT_1440P_50 Intftype:8 timing:19
PanelName:DACOUT_1440P_60 Intftype:8 timing:20
PanelName:VGAOUT_1024x768P_60 Intftype:8 timing:45
PanelName:VGAOUT_1280x1024P_60 Intftype:8 timing:48
PanelName:VGAOUT_1366x768P_60 Intftype:8 timing:46
PanelName:VGAOUT_1440x900P_60 Intftype:8 timing:49
PanelName:VGAOUT_1280x800P_60 Intftype:8 timing:47
PanelName:VGAOUT_1680x1050P_60 Intftype:8 timing:51
PanelName:VGAOUT_1600x1200P_60 Intftype:8 timing:50
PanelName:BT1120OUT_576P_50 Intftype:14 timing:5
PanelName:BT1120OUT_480P_60 Intftype:14 timing:3
PanelName:BT1120OUT_1080P_25 Intftype:14 timing:11
PanelName:BT1120OUT_1080P_30 Intftype:14 timing:12
PanelName:BT1120OUT_1080P_50 Intftype:14 timing:15
PanelName:BT1120OUT_1080P_2997 Intftype:14 timing:13
PanelName:BT1120OUT_1080P_5994 Intftype:14 timing:16
PanelName:BT1120OUT_1080P_60 Intftype:14 timing:18
PanelName:BT1120OUT_720P_50 Intftype:14 timing:7
PanelName:BT1120OUT_720P_5994 Intftype:14 timing:8
PanelName:BT1120OUT_720P_60 Intftype:14 timing:9
PanelName:BT656OUT_576P_50 Intftype:12 timing:5
PanelName:BT656OUT_480P_60 Intftype:12 timing:3
PanelName:BT656OUT_1080P_25 Intftype:12 timing:11
PanelName:BT656OUT_1080P_2997 Intftype:12 timing:13
PanelName:BT656OUT_1080P_30 Intftype:12 timing:12
PanelName:BT656OUT_720P_50 Intftype:12 timing:7
PanelName:BT656OUT_720P_60 Intftype:12 timing:9
PanelName:BT656OUT_720P_5994 Intftype:12 timing:8
pq table size:25892
DBC Value=
    0,0,0
    0,0,0
    0,0,0
start dump [motion_table](0, 0)
end dump
start dump [motion_hdmi_dtv_table](0, 0)
end dump
start dump [motion_comp_pc_table](0, 0)
end dump
start dump [misc_param_table](0, 0)
end dump
start dump [misc_luma_table](0, 0)
end dump
start dump [noise_table](0, 0)
end dump
start dump [misc_table](0, 0)
end dump

```

• 调试信息分析

`./config/dump_config` 得到的是 `/proc/mi_modules/common/` 里各文件的信息，含panel size、DBC Value、motion_table、motion_hdmi_dtv_table、motion_comp_pc_table、misc_param_table、misc_luma_table、noise_table、misc_table等。

• 参数分析

参数		描述
panel	PanelName	LCD屏名称
	Intftype	LCD屏接口类型: E_MAPI_PNL_INTF_TTL = 0, ///< TTL type E_MAPI_PNL_INTF_LVDS, ///< LVDS type E_MAPI_PNL_INTF_RSDS, ///< RSDS type E_MAPI_PNL_INTF_MINILVDS, ///< TCON ///< Analog TCON E_MAPI_PNL_INTF_ANALOG_MINILVDS, ///< Digital TCON E_MAPI_PNL_INTF_DIGITAL_MINILVDS, E_MAPI_PNL_INTF_MFC, ///< Ursa (TTL output to Ursa) E_MAPI_PNL_INTF_DAC_I, ///< DAC output E_MAPI_PNL_INTF_DAC_P, ///< DAC output ///< For PDP(Vsync use Manually MODE) E_MAPI_PNL_INTF_PDPLVDS, E_MAPI_PNL_INTF_EXT, ///< EXT LPLL TYPE E_MAPI_PNL_INTF_MIPI_DSI, E_MAPI_PNL_INTF_BT656, E_MAPI_PNL_INTF_BT601, E_MAPI_PNL_INTF_BT1120, ///< BT1120 E_MAPI_PNL_INTF_MCU_TYPE, ///< MCU Type E_MAPI_PNL_INTF_SRGB, ///< sRGB E_MAPI_PNL_INTF_TTL_SPI_IF,
	timing	LCD屏显示输出时序： DISPLAYTIMING_MIN = 0, DISPLAYTIMING_DACOUT_DEFAULT = 0, DISPLAYTIMING_DACOUT_FULL_HD, DISPLAYTIMING_DACOUT_480P

```
DISPLAYTIMING_DACOUT_480I,  
DISPLAYTIMING_DACOUT_480P,  
DISPLAYTIMING_DACOUT_576I,  
DISPLAYTIMING_DACOUT_576P, // 5  
DISPLAYTIMING_DACOUT_720P_2997,  
DISPLAYTIMING_DACOUT_720P_50,  
DISPLAYTIMING_DACOUT_720P_5994,  
DISPLAYTIMING_DACOUT_720P_60,  
DISPLAYTIMING_DACOUT_1080P_24,  
DISPLAYTIMING_DACOUT_1080P_25,  
DISPLAYTIMING_DACOUT_1080P_30, // 12  
DISPLAYTIMING_DACOUT_1080P_2997,  
DISPLAYTIMING_DACOUT_1080I_50,  
DISPLAYTIMING_DACOUT_1080P_50,  
DISPLAYTIMING_DACOUT_1080P_5994,  
DISPLAYTIMING_DACOUT_1080I_60,  
DISPLAYTIMING_DACOUT_1080P_60,  
DISPLAYTIMING_DACOUT_1440P_50,  
DISPLAYTIMING_DACOUT_1440P_60,  
DISPLAYTIMING_DACOUT_1470P_24,  
DISPLAYTIMING_DACOUT_1470P_30,  
DISPLAYTIMING_DACOUT_1470P_60,  
DISPLAYTIMING_DACOUT_2205P_24,  
// For 2k2k  
DISPLAYTIMING_DACOUT_2K2KP_24, // 23  
DISPLAYTIMING_DACOUT_2K2KP_25,  
DISPLAYTIMING_DACOUT_2K2KP_30,  
DISPLAYTIMING_DACOUT_2K2KP_60,  
// For 4k0.5k  
DISPLAYTIMING_DACOUT_4K540P_240,  
// For 4k1k  
DISPLAYTIMING_DACOUT_4K1KP_30,  
DISPLAYTIMING_DACOUT_4K1KP_60,  
DISPLAYTIMING_DACOUT_4K1KP_120,  
// For 4k2k  
DISPLAYTIMING_DACOUT_4K2KP_24,  
DISPLAYTIMING_DACOUT_4K2KP_25,  
DISPLAYTIMING_DACOUT_4K2KP_30, // 33  
DISPLAYTIMING_DACOUT_4K2KP_50,  
DISPLAYTIMING_DACOUT_4K2KP_60,  
DISPLAYTIMING_DACOUT_4096P_24,  
DISPLAYTIMING_DACOUT_4096P_25,  
DISPLAYTIMING_DACOUT_4096P_30,  
DISPLAYTIMING_DACOUT_4096P_50,  
DISPLAYTIMING_DACOUT_4096P_60,  
// For VGA OUTPUT
```

	DISPLAYTIMING_VGAOUT_640x480P_60, DISPLAYTIMING_VGAOUT_1280x720P_60, DISPLAYTIMING_VGAOUT_1024x768P_60, DISPLAYTIMING_VGAOUT_1366x768P_60, DISPLAYTIMING_VGAOUT_1280x800P_60, DISPLAYTIMING_VGAOUT_1280x1024P_60, DISPLAYTIMING_VGAOUT_1440x900P_60, DISPLAYTIMING_VGAOUT_1600x1200P_60, DISPLAYTIMING_VGAOUT_1680x1050P_60, DISPLAYTIMING_VGAOUT_1920x1080P_60, // 43 // For TTL output DISPLAYTIMING_TTLOUT_480X272_60, DISPLAYTIMING_DACOUT_4K2KP_120, DISPLAYTIMING_USER, DISPLAYTIMING_MAX_NUM,
DBC Value	暂时无效果
motion_table	暂时无效果
motion_hdmi_dtv_table	暂时无效果
motion_comp_pc_table	暂时无效果
misc_param_table	暂时无效果
misc_luma_table	暂时无效果
noise_table	暂时无效果
misc_table	暂时无效果



5.2. mi_log_info

5.2.1. cat

- 调试信息


```
# cat /proc/mi_modules/mi_log_info

----- Log Path -----
log path:
----- Store Path -----
store path: /mnt
----- Module Log Level -----
Log module      Level
-----
mi_ive           2(WRN)
mi_vdf           2(WRN)
mi_venc          2(WRN)
mi_rgn           2(WRN)
mi_ai            2(WRN)
mi_ao            2(WRN)
mi_vif           2(WRN)
mi_vpe           2(WRN)
mi_vdec          2(WRN)
mi_sys           2(WRN)
mi_fb            2(WRN)
mi_hdmi          2(WRN)
mi_divp          2(WRN)
mi_gfx           2(WRN)
mi_vdisp         2(WRN)
mi_disp          2(WRN)
mi_os            2(WRN)
mi_iae           2(WRN)
mi_md            2(WRN)
mi_od            2(WRN)
mi_shadow        2(WRN)
```

- 调试信息分析

说明了默认的 **log path** 或 **store path**，同时说明了各模块的debug_level的值。

- 参数分析

参数		描述
mi_log_info	log path	暂时无效果
	store path	暂时无效果
	Log module	各个模块的名称
	Level	打印等级

5.2.2. echo

功能	修改模块debug_level
命令	echo [ModID]=[Level] > /proc/mi_modules/mi_log_info
参数说明	[ModID] 模块的名字 mi_ive mi_vdf mi_venc mi_rgn mi_ai mi_ao mi_vif mi_vpe mi_vdec mi_sys mi_fb mi_hdmi mi_divp mi_gfx mi_vdisp mi_disp mi_os mi_iae mi_md mi_od mi_shadow [Level] 0 无Debug信息 (MI_DBG_NONE) 1 只显示error的信息 (MI_DBG_ERR) 2 显示level<=2的信息 (MI_DBG_WRN) 3 显示level<=3的信息 (MI_DBG_API) 4 显示level<=4的信息 (MI_DBG_KMSG) 5 显示level<=5的信息 (MI_DBG_INFO) 6 显示level<=6的信息 (MI_DBG_DEBUG) 7 显示 level<=7的信息 (MI_DBG_TRACE) 8 显示所有信息 (MI_DBG_ALL)
举例	echo mi_sys=2 > /proc/mi_modules/mi_log_info mi_sys 的debug_level修改为2

功能	修改log的路径
命令	echo log=[Path] > /proc/mi_modules/mi_log_info
参数说明	[Path] 路径
举例	echo log=/mnt > /proc/mi_modules/mi_log_info

功能	修改存储log的路径
命令	echo storepath=[Path] > /proc/mi_modules/mi_log_info
参数说明	[Path] 路径
举例	echo storepath=/mnt > /proc/mi_modules/mi_log_info

5.3. mi_global_info

5.3.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_global_info

miu_and_lx_info:
ARM_MIU0_BUS_BASE 0x20000000      ARM_MIU0_BASE_ADDR 0x0
ARM_MIU1_BUS_BASE 0x60000000      ARM_MIU1_BASE_ADDR
0x80000000
ARM_MIU2_BUS_BASE 0xffffffff      ARM_MIU2_BASE_ADDR
0xffffffff
lx_mem_addr 0x20c00000      lx_mem_size 0xe300000
lx_mem2_addr 0xffffffff      lx_mem2_size 0xffffffff
lx_mem3_addr 0xffffffff      lx_mem3_size 0xffffffff

KernelProtect IP white list:
clientId                      name
43                            MIU_CLIENT_MIPS_RW
```

```

50 MIU_CLIENT_NAND_RW
82 修改模块debug_level
MIU_CLIENT_USB_UHC0_RW
83 MIU_CLIENT_USB_UHC1_RW
84 MIU_CLIENT_USB_UHC2_RW
18 MIU_CLIENT_G3D_RW
140 MIU_CLIENT_USB3_RW
129 MIU_CLIENT_SDIO_RW
165 MIU_CLIENT_SATA_RW
133 MIU_CLIENT_USB_UHC3_RW
225 MIU_CLIENT_USB30_1_RW
226 MIU_CLIENT_USB30_2_RW
5 MIU_CLIENT_BDMA_RW
14 MIU_CLIENT_EMAC_RW

PAGE_OFFSET - the virtual address of the start of the kernel
image
TASK_SIZE - the maximum size of a user space task
PAGE_OFFSET TASK_SIZE VMALLOC_START VMALLOC_END
c0000000 bf000000 c4000000 ff800000
Sigmastar Module mi_sys version: project_commit.2ba039d
sdk_commit.c06f902 build_time.20210717175137

```

- 调试信息分析

该调试信息提供了全局global的一些信息。

- 参数分析

参数	描述	
miu_and_lx_info (以只有一个MIU为例)	ARM_MIU0_BUS_BASE	Miu0 bus base
	ARM_MIU0_BASE_ADDR	Miu0 base addr
	lx_mem_addr	Linux镜像占的memory的起始地址 (属于cpu bus address)
	lx_mem_size	Linux镜像占的memory的size
kernelProtect IP white list	clientId	Miu protect的IP白名单里的IP的id (从未分group的角度看的全局的id)
	name	与clientId对应的该IP的实际的名字
Sigmastar Module mi_sys version	project_commit	project对应的commit
	sdk_commit	sdk对应的commit
	build_time	sdk的build时间

5.4. mi_bufqueue_status

5.4.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_bufqueue_status

dump Queues in input port only for enabled port:
ModId    DevId    ChnId    PassId    InPortId    UsrInjectQ_cnt
BindInQ_cnt
12        0         0         0         0         0
0

dump Queues in output port only for enabled port:
```

ModId	DevId	ChnId	PassId	OutPortId	DrvBkRefFifoQ_cnt
DrvBkRefFifoQ_size	UsrGetFifoQ_cnt				
12	0	0	0	0	0
0		0			

- 调试信息分析
Dump当前各modules的enable input/output port 相关Queue的信息。
- 参数分析

参数		描述
dump Queues in input port only for enabled port	ModId	该input port所在的module id
	DevId	该input port所在的device id
	ChnId	该input port所在的channel id
	PassId	该input port所在的pass id
	InPortId	该input port的id
	UsrInjectQ_cnt	该 InputPort中 UsrInjectBufQueue 里的 buffer 数目。
	BindInQ_cnt	该 InputPort中 UsrPipeInBufQueue 里的 buffer 数目。
dump Queues in output port only for enabled port	ModId	该output port所在的 module id
	DevId	该output port所在的device id
	ChnId	该output port所在的 channel id
	PassId	该output port所在的pass id

PassId	该output port所在的pass id
OutPortId	该output port的id
DrvBkRefFifoQ_cnt	该OutputPort的 DrvBkRefFifoQueue里的 buffer数目
DrvBkRefFifoQ_size	该OutputPort的 DrvBkRefFifoQueue里的 buffer的总size
UsrGetFifoQ_cnt	该OutputPort的 UsrGetFifoBufQueue里的 buffer数目

5.5. mi_dump_buffer_delay_worker

5.5.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_dump_buffer_delay_worker

delay_worker_id  module_name  force_stop  dev_id  chn_id
port_type  port_id  Queue_name
0          mi_disp      0          0          0
inport      0          BindInput
stored_dir  dump_method  dump_method_value
/mnt        bunfnum      2
```

- 调试信息分析

/proc/mi_modules/mi_dump_buffer_delay_worker 文件是和 mi_modulename devid 相连的一个概念，dump device 里的 Queue 里的 buffer 采用的是 delay worker 的方式实现的，可以通过对 mi_dump_buffer_delay_worker 的 cat 操作查看还有哪些 delay worker 正在进行中，echo force_stop_dump delay_worker_id 强制结束指定的一个 delay worker 过程。

- 参数说明

参数	描述
delay_worker_id	该 delay worker 的 id。如果 delay worker 完成的话，该 id 会回收，给后续其他的 delay worker 作为 id 使用。
module_name	该 delay worker 所对应的 module 的 id。
force_stop	该 delay worker 当前是否处于强制结束阶段，1 表示已经被设置了强制结束，0 表示未被设置。
dev_id	该 delay worker 所对应的 device id。
chn_id	该 delay worker 所对应的 channel id。
port_type	该 delay worker 所对应的 port 的 type，是 input port 或 outputport。
port_id	该 delay worker 所对应的 port 的 id。
Queue_name	该 delay worker 所对应的 Queue 的 name。
stored_dir	该 delay worker 所对应的要 dump 的 data 所要存放的绝对路径（不含最终存放的文件名）。
dump_method	该 delay worker 所对应的 dump 方法: bufnum 或 time 或 start。
dump_method_value	Dump 方法所对应的值。

5.5.2. echo

功能	强制结束 delay worker
命令	echo force_stop_dump [WorkID] >/proc/mi_modules/mi_dump_buffer_delay_worker
参数说明	[WorkID] : delay_work 的 ID
举例	echo force_stop_dump 0 > /proc/mi_modules/mi_dump_buffer_delay_worker 强制停止 delay_work 0

5.6. mi_infer_graph.py

5.6.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_infer_graph.py
```

```
/ # cat /proc/mi_modules/mi_infer_graph.py
"""
    Sigmastar mi_sys infergraph debug utility.
    please run it on linux with python installed.
    Please run 'sudo apt-get install python-pydot' to make sure python-pydot been i
nstalled at first.
"""
try:
    # Try to load pydotplus
    import pydotplus as pydot
except ImportError:
    import pydot

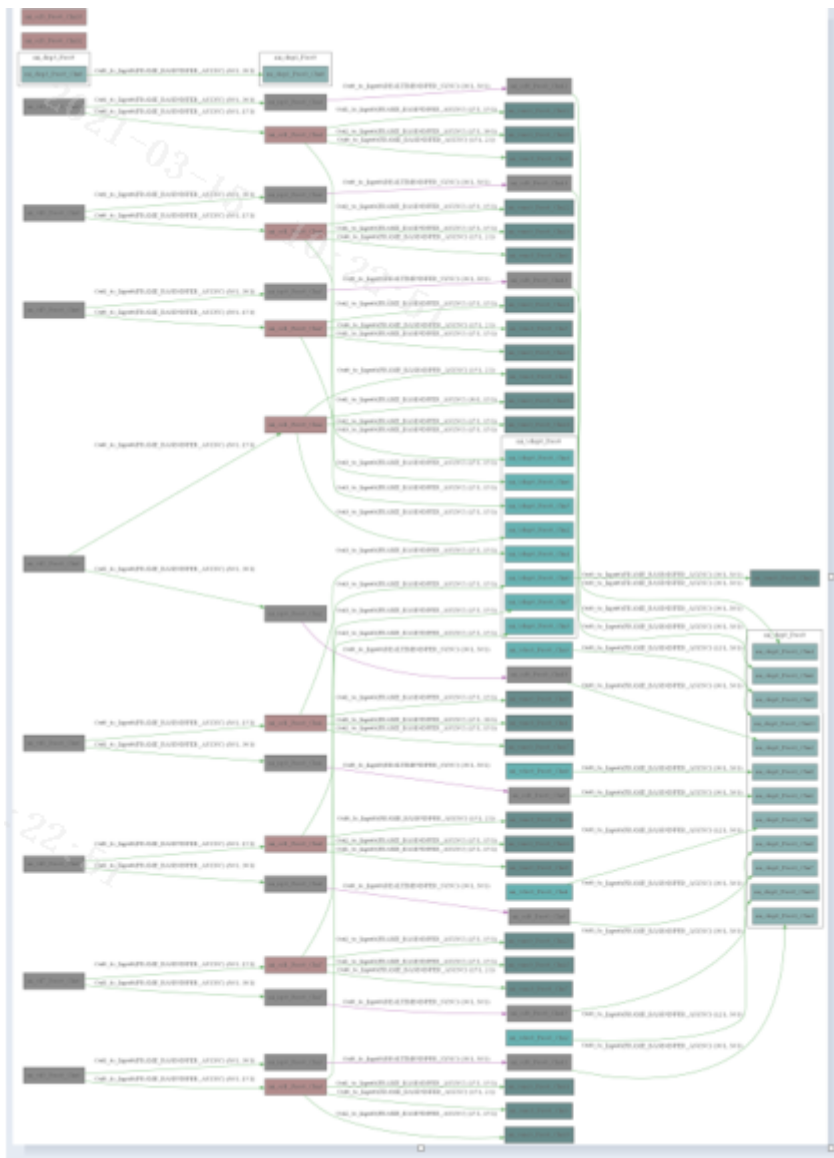
def main():
    file_name = __file__
    base_name = file_name[file_name.find('.')]
    output_filename = base_name+"_"+mi_sys_infer_DAG.png"
    pydot_graph = pydot.Dot("infer_graph", graph_type='digraph', rankdir='LR')
    pydot_nodes = {}
    for node in pydot_nodes.values():
        pydot_graph.add_node(node)

    ext = output_filename[output_filename.rfind('.')+1:]

    with open(output_filename, 'wb') as fid:
        fid.write(pydot_graph.create(format=ext))
```

- 调试信息分析

sys提供proc fs生成运行时信息的python脚本，可以将其cat出来保存到linux server上，然后将其拷贝到文本里，然后在Linux server上使用pyhton2运行，会输出两张图片，默认的图片格式是简化版，一张是“infer_DAG”，一张是“perf_static”，其中一张输出的图片如下：



矩形框中表示的是 channel pass name ；

线上的label表示的是从哪个input port到哪个output port，以及bind type，和 source frame rate、dest frame rate。

5.7. mi_graph_contrl

5.7.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_graph_ctrl

/ # cat /proc/mi_modules/mi_graph_ctrl
dump bind off
```

- 调试信息分析

上述命令的“dump bind”字段表示mi_infer_graph.py脚本输出的样式，“off”表示输出简化绑定关系图，“on”表示输出详细绑定关系图。

5.7.2. echo

功能	控制python脚本输出的图片样式
命令	echo [command] >/proc/mi_modules/mi_graph_ctrl
参数说明	[command] 命令: dump bind on:控制输出详细绑定关系图 dump bind off:控制输出简化绑定关系图
举例	输出详细绑定关系图： # echo dump bind on > /proc/mi_modules/mi_graph_ctrl # cat /proc/mi_modules/mi_infer_graph.py

5.8. mi_deadlock_info

5.8.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_deadlock_info

thread_name:divp0_P0_MAIN pid:601
owned_lock_name:mi_sys_global_devLastCmdqLock
```

```
caller_func_name:mi_sys_internal_main_worker_thread
caller_func_line:10157
```

- 调试信息分析

打印出可能造成deadlock的位置。

- 参数说明

参数		描述
deadlock Info	thread_name	造成deadlock的线程名字
	pid	线程的task pid
	owned_lock_name	锁的名字
	caller_func_name	拿锁的function name
	caller_func_line	拿锁的具体位置

5.9. debug_level

5.9.1. cat/echo

- 调试信息

```
# cat /proc/mi_modules/mi_sys/debug_level 2
```

- 调试信息分析

为了控制打印级别，每个module（包括sys）都有各自的debug level，分别由/proc/mi_modules/mi_modulename/debug_level控制，其中disp, divp, rgn等为modulename。上面以/proc/mi_modules/mi_sys/debug_level为例。

功能	打印debug警告级别
命令	cat /proc/mi_modules/[ModuleName]/debug_level
参数说明	[ModuleName] 模块的名字 mi_disp mi_gfx mi_rgn mi_vdec mi_vpe mi_ai mi_divp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif
举例	cat /proc/mi_modules/mi_sys/debug_level 2 mi_sys 的debug警告级别为2(显示warning和error的信息)

功能	修改警告级别
命令	echo [Level] > /proc/mi_modules/[ModuleName]/debug_level
参数说明	[Level] 0 无Debug信息 (MI_DBG_NONE) 1 只显示error的信息 (MI_DBG_ERR) 2 显示level<=2的信息 (MI_DBG_WRN) 3 显示level<=3的信息 (MI_DBG_API) 4 显示level<=4的信息 (MI_DBG_KMSG) 5 显示level<=5的信息 (MI_DBG_INFO) 6 显示level<=6的信息 (MI_DBG_DEBUG) 7 显示 level<=7的信息 (MI_DBG_TRACE) 8 显示所有信息 (MI_DBG_ALL) [ModuleName] 模块的名字 mi_disp mi_gfx mi_rgn mi_vdec mi_vpe mi_ai mi_divp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif

5.10. debug_file

5.10.1. echo

- 调试信息

```
# echo mi_sys_impl.c > /proc/mi_modules/mi_sys/debug_file
```

- 调试信息分析

用于debug时使能文件mi_sys_impl.c里的所有level等级的DBG log信息都可以在串口打印。

- 参数说明

参数	描述
命令	echo [fileName] ... > /proc/mi_modules/mi_xxx/debug_file 表示运行时使能打印函数fileName里的所有level等级的DBG log信息，可以同时使能多个函数，函数之间用空格隔开。 echo > /proc/mi_modules/mi_xxx/debug_file 表示关闭之前已打开的函数fileName的所有level等级的打印功能，此时函数fileName里的DBG log打印只能按/proc/mi_modules/mi_xxx/debug_level设置的等级打印。 注：fileName必须属于对应的mi_xxx模块。
参数说明	[mi_xxx]：表示MI模块名称，可取： mi_disp mi_gfx mi_rgn mi_vdec mi_vpe mi_ai mi_divp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif

5.11. debug_func

5.11.1. echo

- 调试信息

```
# echo MI_SYS_IMPL_Init > /proc/mi_modules/mi_sys/debug_func
```

- 调试信息分析

用于debug时使能函数MI_SYS_IMPL_Init()里的所有level等级的DBG log信息都可以在串口打印。

- 参数说明

参数	描述
命令	echo [funcName] ... > /proc/mi_modules/mi_xxx/debug_func 表示运行时使能打印函数funcName里的所有level等级的DBG log信息，可以同时使能多个函数，函数之间用空格隔开。 echo > /proc/mi_modules/mi_xxx/debug_func 表示关闭之前已打开的函数funcName的所有level等级的打印功能，此时函数funcName里的DBG log打印只能按/proc/mi_modules/mi_xxx/debug_level设置的等级打印。 注：funcName必须属于对应的mi_xxx模块。
参数说明	[mi_xxx]：表示MI模块名称，可取： mi_disp mi_gfx mi_rgn mi_vdec mi_vpe mi_ai mi_divp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif

5.12. module_version_file

5.12.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_ai/module_version_file

MStar Module version: project_commit.sdk_commit.build_time
c1799df.fd2d52b.20171225180033

# cat /proc/mi_modules/mi_global_info

...

MStar Module version: project_commit.sdk_commit.build_time
cb68bfd.44aca45.20171226100257

...
```

- 调试信息分析

xxxx_version_file提供了version信息，/proc/mi_modules/mi_global_info里我们也提供了version信息，不过其本质上指的是mi_sys模块的version信息，上

面只是以mi_vi 和 mi_global_info 为例子，每个模块都有自己对应的version file。

- 参数说明

参数		描述
Version Info	project_commit	模块编译ko时的整包project commit，如果单独替换ko，模块基于的commit有变化，ko version里得到的commit也会变更。
	sdk_commit	模块编译ko时的整包sdk commit，如果单独替换ko，模块基于的commit有变化，ko version里得到的commit也会变更。
	build_time	实际的build时间，精确到秒，即使make clean;make image整体来build的话，各个ko的时间也会有差别。

5.13. show threads

5.13.1. echo

- 调试信息

```
# echo show_threads > /proc/mi_modules/mi_sys/mi_sys0

<6>    task                PC stack      pid
father
<6>busybox                S            1            0 0x00000000
<3>[<c01d3a6f>] (__schedule) from [<c01d3bb7>]
(schedule+0x57/0x64)
<3>[<c01d3bb7>] (schedule) from [<c001bfbd>]
(do_wait+0xed/0x134)
<3>[<c001bfbd>] (do_wait) from [<c001c367>]
(SyS_wait4+0x69/0x7e)
<3>[<c001c367>] (SyS_wait4) from [<c000d2a1>]
(ret_fast_syscall+0x1/0x54)
<6>kthreadd                S            2            0 0x00000000
<3>[<c01d3a6f>] (__schedule) from [<c01d3bb7>]
(schedule+0x57/0x64)
<3>[<c01d3bb7>] (schedule) from [<c002b077>]
```



```

(kthreadd+0x6d/0xf6)
<3>[<c002b077>] (kthreadd) from [<c000d381>]
(ret_from_fork+0x11/0x30)
<6>ksoftirqd/0 S 3 2 0x00000000
<3>[<c01d3a6f>] (__schedule) from [<c01d3bb7>]
(schedule+0x57/0x64)
<3>[<c01d3bb7>] (schedule) from [<c002c9af>]
(smpboot_thread_fn+0xff/0x12c)
<3>[<c002c9af>] (smpboot_thread_fn) from [<c002a911>]
(kthread+0xa1/0xb4)
<3>[<c002a911>] (kthread) from [<c000d381>]
(ret_from_fork+0x11/0x30)
.....
.....
<6>du R running task
3641 3640 0x00000000
<3>[<c01d3a6f>] (__schedule) from [<c01d3c5b>]
(preempt_schedule_common+0x1b/0x28)
<3>[<c01d3c5b>] (preempt_schedule_common) from [<c00a0c7f>]
(__find_get_block+0x159/0x176)
<3>[<c00a0c7f>] (__find_get_block) from [<c00a1013>]
(__getblk_gfp+0x11/0x1e2)
<3>[<c00a1013>] (__getblk_gfp) from [<c00a1a4d>]
(__bread_gfp+0x7/0x86)

```

- 调试信息分析

查看当前所有线程的 **call stack** 状态。

- 该命令可以帮组查询&定位死锁问题。
- 该命令打印较多，最好放在kmsg中打印，以免被其他信息污染。

- 参数说明

参数		描述
show_threads	task	线程名p->comm
	state	线程运行状态
	pid	线程的pid
	father	线程的父进程id
	flags	low level flags

5.14. debug_frc

5.14.1. echo

- 调试信息

```
# echo debug_frc on 12 0 0 0 0 "out" >
/proc/mi_modules/mi_sys/mi_sys0
```

- 调试信息分析

printk in _MI_SYS_IMPL_Common_WriteProc ·

Switch enable debug FRC, Modid:12, Dev:0, Chn:0, Pass:0, Port:0,
BufType:1 · 使能帧率控制debug开关

- 参数说明

参数	描述
命令	Switch enable debug FRC: echo debug_frc on [Modid] [Devid] [Chnid] [Passid] [Portid] [in/out] > /proc/mi_modules/mi_sys/mi_sys0 打开帧率控制debug开关 Switch disable debug FRC: echo debug_frc off > /proc/mi_modules/mi_sys/mi_sys0 关闭帧率控制debug开关
参数说明	[Modid] : module id [Devid] : device id [Chnid] : channel id [Passid] : pass id [Portid] : port id [in/out] : input/output方向

5.15. set_outputport_depth

5.15.1. echo

- 调试信息

```
# echo set_outputport_depth 6 0 0 0 0 0 2 4 >
/proc/mi_modules/mi_sys/mi_sys0
```

- 参数说明

参数	描述
命令	Set output port depth: echo set_ouputport_depth [Modid] [Devid] [Chnid] [Passid] [Portid] [u32userFrameDepth] [u32BufQueueDepth] > /proc/mi_modules/mi_sys/mi_sys0
参数说明	[Modid] : module id [Devid] : device id [Chnid] : channel id [Passid] : pass id [Portid] : port id [u32userFrameDepth]: 设置该output用户可以拿到的buf最大数量 [u32BufQueueDepth]: 设置该output系统buf最大数量

5.16. set_thread_priority

5.16.1. echo

- 调试信息

```
# echo set_thread_priority 4 0 99 >
/proc/mi_modules/mi_sys/mi_sys0
# echo set_thread_priority 8 849 99 >
/proc/mi_modules/mi_sys/mi_sys0
```

- 参数说明

参数	描述
命令	Set work_thread_priority by module id: echo set_thread_priority [Modid] [Devid] [u32Priority] > /proc/mi_modules/mi_sys/mi_sys0 set_thread_priority by pid: echo set_thread_priority 8 [pid] [u32Priority] > /proc/mi_modules/mi_sys/mi_sys0
参数说明	[Modid] : module id [Devid] : device id [u32Priority] : 需要设定的线程等级 [pid] : 线程pid

5.17. mmu debug

5.17.1. echo

- 调试信息

```
# echo debug_mmu debug_log 1 1 1 33,34 >  
/proc/mi_modules/mi_sys/mi_sys0
```

- 参数说明

参数	描述
命令	Enable/disable alloc/free buf log: echo debug_mmu debug_log [enable/disable] [enable/disable free] [enable/disable alloc] [module id list] > /proc/mi_modules/mi_sys/mi_sys0
参数说明	[enable/disable] :enable or disable [module id list]: module list, Separated by commas

- 举例

```
1.echo debug_mmu debug_log 1          --- enable alloc/free
buf log
2.echo debug_mmu debug_log 1 0        --- enable alloc buf log
3.echo debug_mmu debug_log 1 1 0      --- enable free buf log
4.echo debug_mmu debug_log 1 1 1 4,6  --- enable ai & vif
alloc/free buf log
5.echo debug_mmu debug_log 1 0 1 4,6  --- enable ai & vif
alloc buf log
6.echo debug_mmu debug_log 1 1 0 4,6  --- enable ai & vif
free buf log
```

5.17.2. insmod

- 调试信息

```
# insmod /config/modules/4.9.227/mi_sys.ko bDebugMmu=1
```

- 参数说明

参数	描述
命令	For debug memory access out of bounds: insmod /config/modules/4.9.227/mi_sys.ko bDebugMmu=1/bDebugMmu=0 # Depends on enable mmu
参数说明	bDebugMmu=1 :enable bDebugMmu=0 :disable default disable

5.18. enable_backtrace

5.18.1. echo

- 调试信息

```
# echo enable_backtrace 1 > /proc/mi_modules/mi_sys/mi_sys0
```

- 调试信息分析

用于串口输出卡住时，可以使用该命令打印当前工作线程运行的backtrace信息。

- 参数说明

参数	描述
命令	<code>echo enable_backtrace 1 > /proc/mi_modules/mi_sys/mi_sys0</code> 串口输出卡住时，使能打印当前工作线程的backtrace信息 <code>echo enable_backtrace 0 > /proc/mi_modules/mi_sys/mi_sys0</code> 串口输出卡住时，关闭打印当前工作线程的backtrace信息

5.19. set_inputport_pattern

5.19.1. echo

- 调试信息

```
# echo set_inputPort_pattern 33 0 0 0 0 1 > /proc/mi_modules/mi_sys/mi_sys0
```

- 调试信息分析

设置使能ModId33 DevId0 ChnId0 PassId0 InputPortId0，此时pipeline中ModId33模块只会保留有一张buf数据向后级模块传递数据（ModId33与后级模块的bindtype需要设置为frame mode），用于debug定位是哪个ModId的buf数据有问题。

- 参数说明

参数	描述
命令	<code>echo set_inputport_pattern [ModId] [Devid] [ChnId] [PassId] [InputPortId] [bEnable] > /proc/mi_modules/mi_sys/mi_sys0</code> 设置ModId的某个inputPort只保留一张buf数据，来向后级ModId传递数据（该ModId与后级ModId的bindType需要设置为frame mode），用于debug定位是哪个ModId的buf数据有问题。
参数说明	[ModId] : module id. [Devid] : device id. [ChnId] : channel id. [PassId] : pass id. [inputPortId] : inputPort Id. [bEnable] : 是否使能，0/1.

5.20. mi_sys0

5.20.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_sys/mi_sys0
```

BinDeerInputPortList																			
chnid	passid	portid	enable	bind_module_id	bind_module_name	bind_devid	bind_chnid	bind_portid	bind_type	bind_param	enable								
chn pre	suc	busy	drop	Enq	run	done	retry	0	drop	0	bar	checkin	run	0	done	retry	0	drop	0
0	472	sea	0	46d	0	0	0	0	0	0	46d	0	0	0	0	0	0	0	0
chn pre	suc	busy	drop	Enq	run	done	retry	0	drop	0	bar	checkin	run	0	done	retry	0	drop	0
0	46a	0	0	46a	0	0	0	0	0	0	46a	0	0	0	0	0	0	0	0
chn pre	suc	busy	drop	Enq	run	done	retry	0	drop	0	bar	checkin	run	0	done	retry	0	drop	0
0	46a	0	0	46a	0	0	0	0	0	0	46a	0	0	0	0	0	0	0	0
chn pre	suc	busy	drop	Enq	run	done	retry	0	drop	0	bar	checkin	run	0	done	retry	0	drop	0
0	46a	0	0	46a	0	0	0	0	0	0	46a	0	0	0	0	0	0	0	0
chn pre	suc	busy	drop	Enq	run	done	retry	0	drop	0	bar	checkin	run	0	done	retry	0	drop	0
0	45f	0	0	45f	0	0	0	0	0	0	45c	3	checkin	run	3	3	retry	0	drop
chn pre	suc	busy	drop	Enq	run	done	retry	0	drop	0	bar	checkin	run	0	done	retry	0	drop	0
31	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- 调试信息分析

该命令显示了当前设备pipeline时，各modId模块数据流buf处理的相关信息。

- 参数说明

参数	描述
(mod-dev-chn info)	module module Id，取值：mi_disp mi_gfx mi_rgn mi_vdec mi_vpe mi_ai mi_divp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar

	mi_vif	
	devId	device Id
	chn	channel Id
pre (咨询当前modId是否可以处理buf的handleinfo)	suc	当前modId已成功预处理buf的次数
	busy	当前modId处于busy的次数
	drop	当前modId对预处理的buf进行drop操作的次数
Enq (将buf添加到当前modId的队列中的handleinfo)	run	当前modId enqueue操作返回处于running状态的次数
	done	当前modId已处理完成queue里buf的次数
	retry	当前modId重新尝试处理queue中buf的次数
	drop	当前modId将queue中待处理的buf drop掉的次数
	bar	cmdq相关
checkin (inputport buf handleinfo)	run	当前modId处理inputport buf操作返回处于running状态的次数
	done	当前modId已处理完成inputport里的buf个数
	retry	当前modId重新尝试处理inputport buf的系数
	drop	当前modId将待处理的inputport buf drop掉的次数
checkout (outputport buf handleinfo)	run	当前modId处理outputport buf操作返回处于running状态的次数

done	当前modId已处理完成outputport里buf的次数
retry	当前modId重新尝试处理outputport buf的次数
drop	当前modId将待处理的outputport buf drop掉的次数

5.21. miu_protect

5.21.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/miu_protect

===== start miu_protect_info =====
kernel protect enabled
LX :
cpu_start_addr:0x20c00000    size:0xe300000
miu_index    miuBlockIndex    start_cpu_bus_pa    length
0x0          0x00          0x20c00000          0x460000
KernelProtect IP white list:
    clientId          name
    43                MIU_CLIENT_MIPS_RW
    50                MIU_CLIENT_NAND_RW
    82                MIU_CLIENT_USB_UHC0_RW
    83                MIU_CLIENT_USB_UHC1_RW
    84                MIU_CLIENT_USB_UHC2_RW
    18                MIU_CLIENT_G3D_RW
    140               MIU_CLIENT_USB3_RW
    129               MIU_CLIENT_SDIO_RW
    165               MIU_CLIENT_SATA_RW
    133               MIU_CLIENT_USB_UHC3_RW
    225               MIU_CLIENT_USB30_1_RW
    226               MIU_CLIENT_USB30_2_RW
    5                 MIU_CLIENT_BDMA_RW
    14                MIU_CLIENT_EMAC_RW
```

- 调试信息分析

该命令显示了miu protect相关的信息。

- 参数分析

参数		描述
kernel protect		值是enabled或者disabled，表示是否有enable kernel protect，默认是需要enable kernel protect的。
LX (以只有一个LX为例，LX表示linux 镜像对应的memory)	cpu_start_addr	该LX对应的起始CPU addr。
	size	该LX对应的size。
某个kernel protect 的range的相关信息	miu_index	编号。
	miuBlockIndex	总共4个miu范围的编号信息。
	start_cpu_bus_pa	该range的起始cpu bus addr。
	length	该range的长度。
KernelProtect IP white list	clientId	Miu protect的IP白名单里的IP的id（从未分group的角度看的全局id）。
	name	与clientId对应的该IP的实际的名字。

5.22. mma_heap_name0

5.22.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/mma_heap_name0

mma heap name    heap_base_cpu_bus_addr  length
chunk_mgr_avail  mmuEnable
```

mma_heap_name0	40000000	1f000000	1ecbf000
1			
addsize_before	addsize_after	bMmuDelayUnmap	free_entry_num
0	0	0	3d9
chunk_mgr info:			
	offset	length	avail
HighPeak	LowPeak		used
	0	20000000	1fcbf000
341000	0		341000
each chunk info:			
	offset	length	used_flag
pid	Module		task_name
	0	300000	1
			CMDMEM
-1	mi_sys		
	300000	1000	1
			sys-logConfig
-1	mi_sys		
	301000	40000	1
			sys-logBuffer
-1	mi_sys		
	341000	1fcbf000	0
			NA
-1	mi_sys		

- 调试信息分析

该cat信息对应的是该mma heap的基本信息和当前的一些状态。

- 参数分析

参数	描述
heap basic info	mma heap name
	mma heap的名称。
	heap_base_cpu_bus_addr
	heap的起始cpu bus addr。
	length
	mma heap的整个length。
	chunk_mgr_avail
	heap里的剩余的未用的memory的总量。
	mmuEnable
	使能mmu的开关
	addsize_before
	设置在申请的buf前多申请预留的size大小

	addrsz_after	设置在申请的buf后多申请预留的size大小
	bMmuDelayUnmap	buf free时是否开启延时unmap 当前申请的buf，当下次再被申请时，先unmap再map 1：开启延时unmap 0：关闭延时unmap
	free_entry_num	空闲的entry的个数
chunk_mgr info	offset	chunk mgr的offset,由于整个mma heap做为一个chunk mgr,因此该值永远为0。
	length	chunk mgr管理的整个mma vpa length,由于整个mma heap做为一个chunk mgr，因此当关闭mmu时，该值等于mma heap length；当开启mmu时，该值大于mma heap length。
	avail	chunk mgr(即heap)里的剩余的未用的memory的总量。
	used	chunk mgr(即heap)里已使用的memory的总量。
	HighPeak	统计的一段时间内的内存峰值
	LowPeak	统计的一段时间内的内存谷值
each chunk info (chunk mgr里各chunk的信息和使用情况)	offset	该chunk在chunk mgr里的offset。
	length	该chunk的长度。
	used_flag	该chunk是否被alloc出去使用了。是的话该值为1；否则free状态的话该值为0。
	task_name	如果used flag为1，则task_name里存的是哪个task使

	task_name 正在运行的任务 task 使用的它；否则该值为无效值 NA。
pid	暂时无效
Module	该chunk被哪个Module使用了

5.22.2. echo

功能	即dump 指定offset和length的data到指定的路径
命令	echo [Path] [Offset] [Length] > /proc/mi_modules/mi_sys_mma/mma_heap_name[Num]
参数说明	[Path] 文件要保存的路径,只需提供路径，系统会根据参数自动生成对应的文件名。 [Offset] 从该mma heap的offset开始dump data,必须4KB对齐。 [Length] 在该mma heap里dump的总数据量, 必须4KB对齐 [Num] mma head 对应的数字，目前 mma_heap_name0 可通过 cat /proc/cmdline 查看
举例	echo /mnt/ 0 4096 > /proc/mi_modules/mi_sys_mma/mma_heap_name0 在 /mnt下产生 mma__mma_heap_name0__0__4096.bin 文件

5.23. imi_mma_heap

5.23.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/imi_mma_heap

mma heap name    heap_base_cpu_bus_addr  length
chunk_mgr_avail  mmuEnable
mma_heap_name0   22000000                1f80000    1c8a000
1
chunk_mgr info:
```

		offset	length	avail	
		0	10000000	fd0a000	
		each chunk info:			
		offset	length	used_flag	task_name
pid	Module				
		0	20000	0	NA
-1	mi_sys				

- 调试信息分析

该cat信息对应的是该mma heap的基本信息和当前的一些状态。

- 参数分析

参数		描述
heap basic info	mma heap name	mma heap的名称
	heap_base_cpu_bus_addr	该allocator里的allocations中未被使用的数目。
	length	heap的长度
	chunk_mgr_avail	heap里的剩余的未用的memory的总量
	mmuEnable	Mma heap是否支持HW_MMU。该值为1表示支持，为0表示不支持
chunk_mgr info	offset	chunk mgr的offset,由于整个mma heap做为一个chunk mgr,因此该值永远为0。 allocator里的逻辑offset。
	length	chunk mgr 的 length,由于整个mma heap 做为一个 chunk mgr,因此该值永远为 mma heap length。
	avail	chunk mgr 的 length,由于整个mma heap 做为一个chunk mgr,因此该值永远为 mma heap length。
each chunk info	offset	该chunk在chunk mgr里的offset

(chunk mgr里各
chunk的信息和使
用情况)

length	该chunk的length
used_flag	该chunk是否allocated。是，该值为1；否，该值为0
task_name	如果used flag为1，则task_name存的task使用它；否则该值为无效值NA
pid	该chunk所属的current->tgid
Module	该chunk所属的模块

5.24. meta

5.24.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/meta

===== start meta_info =====
basic info:
total page count:16      each meta data size:256
total_count_with_metadatasize 256
total_free_count_with_metadatasize:254
meta info :
ref_cnt=3
each allocation info:
offset_in_pool  length      phy_addr      va_in_kern
real_used_flag
0x0             0x100      0x2336000     c1736000
0
0x100           0x100      0x2336100     c1736100
0
0x200           0x100      0x2336200     c1736200
0
0x300           0x100      0x2336300     c1736300
0
0x400           0x100      0x2336400     c1736400
0
```



```

0x500      0x100      0x2336500      c1736500
0
0x600      0x100      0x2336600      c1736600
0
0x700      0x100      0x2336700      c1736700
0
0x800      0x100      0x2336800      c1736800
0
0x900      0x100      0x2336900      c1736900
0
0xa00      0x100      0x2336a00      c1736a00
0
.....
=====  end meta_info
=====

```

- 调试信息分析

只要用过meta allocator(只有一个meta allocator)· 就会生成/proc/mi_modules/mi_sys_mma/meta· 且其对应的memory就会完全alloc· 等待被使用。

5.24.2. echo

功能	即dump 指定offset和length的数据到指定的路径
命令	echo [Path] [Offset] [Length] > /proc/mi_modules/mi_sys_mma/vb_pool_global
参数说明	[Path] 文件要保存的路径,只需提供路径· 系统会根据参数自动生成对应的文件名。 [Offset] 从该allocator的哪个offset开始dump data,是allocator的逻辑offset。 [Length]在该allocator里dump的总的数据量的大小。
举例	echo /mnt 0 1024 > /proc/mi_modules/mi_sys_mma/meta 在 /mnt 下产生 meta__0__1024.bin

5.25. mi_modulenamedevid

5.25.1. cat

- 调试信息

```
cat /proc/mi_modules/mi_disp/mi_disp0
```

```
/ # cat /proc/mi_modules/mi_disp/mi_disp0
-----Common info for mi_disp0-----
ChnNum      2  EnChnNum      1  InPortNum      16  OutPortNum      0  CollectSize      0
-----Common info for mi_disp0 only dump enabled chn-----
ChnId      0  EnInPNum      1  EnOutPNum      0  MMAHeapName      (null)
-----Input port common info for mi_disp0 only dump enabled port-----
ChnId      0  PortId      0  SrcFrmRate      30  DstFrmRate      30  user_buf_quota      4  UsrInjectQ_cnt      0  UsrInjectQ_size      0  BindInQ_cnt      0  BindInQ_size      0  WorkingQ_cnt      1  WorkingQ_size      4147200  UsrLockedInjectCnt      0
ChnId      0  PortId      0  bind_module_id      14  bind_module_name      mi_vdisp  bind_Chnid      0  bind_PortId      0
-----Output port common info for mi_disp0 only for enabled port-----
ChnId      0  PortId      0  BufCntQuota      0  UsrLockedCnt      0  totalOutPortInUsed      0  DrvBkRefFifo_cnt      0  DrvBkRefFifo_size      0  UsrGetFifo_cnt      0  UsrGetFifo_size      0  WorkingQ_cnt      0  WorkingQ_size      0
-----BindPeerInputPortList-----
ChnId      0  PortId      0  bind_module_id      14  bind_module_name      mi_vdisp  bind_Chnid      0  bind_PortId      0
```

- 调试信息分析

cat操作的结果分为两部分：

- a. MI_SYS提供的针对该device的通用的调试信息，从device、channel、input port、output port四个角度提供；
- b. 该device私有的信息。私有的信息各module的内容别处会阐述，这里SYS角度只阐述通用的信息。

以cat /proc/mi_modules/mi_vdisp/mi_vdisp0为例，得到上面的结果。

- 参数说明

参数		描述
Common info for device	ChnNum	该device的总的channel 的数目
	EnChnNum	该device的enabled了的channel的数目
	InPortNum	该device的input port的数目
	OutPortNum	该device的output port的数目
	CollectSize	该dev对应的AllocatorCollection含有的allocator的数目总和
Common info for channel(only dump enabled channel)	ChnId	Channel 的id值
	EnInPNum	该channel的enabled input port数量

		EnOutPNum	该channel的enabled output port数量
		MMAHeapName	如果设置了SetChnMMAConf，该值为对应的mma heap name;如果没有，该值为NULL
chn pass pipeline delay in us	Flow	流程点： OnPreProcessInputTask：MI PASS判断是否需要处理当前task，如果可以给output buffer配置； EnqueueInputTask：将input task 交由MI PASS开始处理； BarrierInputTask：插入CMDQ同步指令（比如waitevent，polling engine done）； CheckInputTaskStatus：检查input task是否完成，及完成状态（成功，失败，etc）； DequeueInputTask：通知MI PASS释放input task相关联的资源； OnPollingAsyncOutputTaskConfig：对于输入输出不同步的MI Pass，或者只有输出没有输入的MI Pass，MI SYS查询是否有独立output buf request需求； EnqueueAsyncOutputTask：将非同步输出的output task交由MI PASS做处理； BarrierAsyncOutputTask：对非同步output task插入cmdq同步指令； CheckOutputTaskStatus：检查output task是否完成，及完成状态（成功，失败，etc）； DequeueOutputTask：通知MI PASS释放output task相关联的资源； FinDMADispatch：MI SYS将MI PASS的output task中的buffer传递到绑定的下级MI PASS的input queue。	
	Min	Buffer产生到该MI PASS对应Flow所经过的最小延时。	
	Max	Buffer产生到该MI PASS对应Flow所	

		经过的最大延时。
	Avg	Buffer产生到该MI PASS对应Flow所经过的平均延时。
	Diff	Buffer产生到该MI PASS对应Flow所经过的当前延时。
Input port common info(only dump enabled Input port)	ChnId	该input port所在的channel id
	PortId	该input port的id
	SrcFrmrate	Src帧率
	DstFrmrate	Dst帧率
	user_buf_quota	该InputPort的buff数目的Quota
	UsrInjectQ_cnt	该InputPort 里的UsrInjectBufQueue里的buff数目
	UsrInjectQ_size	该InputPort 里的UsrInjectBufQueue里的buff的总的size
	BindInQ_cnt	该InputPort 里的BindInputBufQueue里的buff数目
	BindInQ_size	该InputPort 里的BindInputBufQueue里的buff的总的size
	WorkingQ_cnt	该InputPort 里的WorkingQueue里的buff数目
	WorkingQ_size	该InputPort 里的WorkingQueue里的buff的总的size
	usrLockedInjectCnt	用户当前拿到了多少个buf
Input port bind info(only dump enabled Input port)	ChnId	该input port所在的channel id
	PortId	该input port的id

	bind_module_id	与该input port 进行了binded的 output port所在的module的id
	bind_module_name	与该input port 进行了binded的 output port所在的module的name
	bind_ChnId	与该input port 进行了binded的 output port所在的channel的id
	bind_PortId	与该input port 进行了binded的 output port的id
Output port common info (only dump enabled Output port)	ChnId	该output port所在的channel的id
	PortId	该output port的id
	BufCntQuota	该OutputPort的 buff数目的Quota
	usrLockedCnt	从UsrGetFifoBufQueue里用户实际拿 走了多少个buffer
	totalOutPortInUsed	totalOutputPortInUsedBuf数目
	DrvBkRefFifoQ_cnt	该OutputPort的DrvBkRefFifoQueue 里的buffer数目
	DrvBkRefFifoQ_size	该OutputPort的DrvBkRefFifoQueue 里的buffer的总size
	UsrGetFifoQ_cnt	该OutputPort的UsrGetFifoBufQueue 里的buffer数目
	UsrGetFifoQ_size	该OutputPort的UsrGetFifoBufQueue 里的buffer的总size
	WorkingQ_cnt	该OutputPort的WorkingQueue里的 buffer数目
	WorkingQ_size	该OutputPort的WorkingQueue里的 buffer的总size
Output port	ChnId	该output port所在的channel的id

BindPeerInputPortList info(only dump enabled Output port)	PortId	该output port的id
	bind_module_id	与该output port 进行了binded的 input port list中的其中一个input port (以下简称该binded input port) 所在的module的id
	bind_module_name	该binded input port所在的module的 name
	bind_ChnId	该binded input port所在的channel id
	bind_PortId	该binded input port的id

5.25.2. echo

功能	获得模块的help信息
命令	echo help > /proc/mi_modules/[ModName]/[ModID]
参数说明	[ModName] 模块的名字 mi_disp mi_gfx mi_rgn mi_vdec mi_vpe mi_ai mi_divp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_bar mi_vif [ModID] 模块中对应的文件一般为模块的名字+数字 如mi_disp0 mi_gfx0 mi_rgn0
举例	echo help > /proc/mi_modules/mi_disp/mi_disp0 得到mi_disp0 节点文件的帮助信息

功能	Dump Port 的信息
命令	echo dump_buffer [ChnID] [PortType] [PortID] [QueueName][Path] [EndMethod] > /proc/mi_modules/[ModName]/[ModID]
参数说明	[ChnID] 通道的ID [PortType] iport 或 oport 分别代表input port output port [PortID] 该port的id [QueueName] 如果是input port，值只能是"UsrInject" 或者"BindInput"，如果是output port，值只能是"UsrGetFifo" [Path] 数据保存的文件路径。注意是绝对路径，系统会根据参数自动生成文件名，不同的buffer保存在不同的文件 [EndMethod] dump buffer的结束方法。仅支持三类： (1) "bufnum=xxx":dump指定数目的buffer (2) "time=xxx"(unit：ms):dump过程持续time对应的时间 (3) "start/end" pair:用start开始dump,用end结束dump,使用时其他5个参数需完全一致。 [ModName] 模块的名字 mi_disp / mi_gfx / mi_rgn / mi_vdec / mi_vpe / mi_ai / mi_divp / mi_shadow / mi_vdisp / mi_ao / mi_hdmi / mi_sys / mi_venc / mi_bar / mi_vif [ModID] 模块中对应的文件，一般为模块的名字+数字
举例	echo dump_buffer 0 iport 0 BindInput /mnt bufnum=1 > /proc/mi_modules/mi_disp/mi_disp0 dump disp模块 iport 的数据

5.26. mem_stat

5.26.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/mem_stat
mi_sys
      CMDMEM          c0000
      sys-logBuffer    40000
      sys-logConfig    1000
      TotalSize        101000
      AllTotalSize     101000
```

- 调试信息分析
- 打印出mi sys 内存状态
- 参数说明

参数		描述
Mem_stat	mi_sys	所属mi_sys模块
	CMDMEM	Cmd的内存大小
	sys-logBuffer	Sys log最大可申请的buf大小
	sys-logConfig	分配（申请）给sys log的buf大小
	TotalSize	ModuleUsdTotalSize 模块使用的size
	AllTotalSize	总的size